

Moteur de tournoi de backgammon — spécification

Document de reconstruction complète (bibliothèque `tournoi`)

26 septembre 2026

Table des matières

1	Objet du document	3
1.1	Conventions de rédaction	4
1.2	Statut	4
2	Principes de conception	4
2.1	Journal d'événements, état dérivé	4
2.2	On ne modifie jamais le journal	4
2.3	Le moteur propose, le TD dispose	5
2.4	Déterminisme du tirage au sort	5
2.5	Aucun départage	5
2.6	Les graphes de matchs sont des données	5
3	Vocabulaire	5
4	Modèle de données	6
4.1	Identifiants et joueurs	6
4.2	Match	6
4.3	Rang	7
4.4	Action	7
4.5	Tirage	8
4.6	État	8
4.7	Graphe de matchs	9
4.8	Sérialisation	10
5	Configuration	10
5.1	Structures	10
5.2	Signification des champs de phase	11
5.3	Validation et valeurs par défaut	12
5.4	Configurations types	13
6	Journal et événements	13
6.1	Le catalogue des codes	13
6.2	Structure d'un événement	14
6.3	Champs utilisés par type	15
6.4	Sémantique de <code>Apply</code>	16
6.4.1	<code>created</code>	16
6.4.2	<code>player_added</code>	16
6.4.3	<code>player_updated</code>	17
6.4.4	<code>player_unavailable</code> et <code>player_available</code>	17
6.4.5	<code>player_withdrawn</code>	17
6.4.6	<code>match_started</code>	18
6.4.7	<code>result</code> et <code>result_corrected</code>	18
6.4.8	<code>match_cancelled</code>	19
6.4.9	<code>bye</code>	19
6.4.10	<code>draw</code>	19

6.4.11	<code>next_phase</code>	19
6.4.12	<code>length_changed</code>	19
6.4.13	<code>config_changed</code>	19
6.4.14	<code>reopened</code>	20
6.4.15	<code>table_changed</code>	20
6.4.16	<code>finished</code>	21
6.4.17	<code>note</code>	21
6.5	Construction d'événements	21
6.6	Boucle d'intégration côté hôte	22
7	Reconstruction de l'état	22
7.1	Replay et New	22
7.2	Entrée dans une phase	22
7.3	Passage d'une phase à la suivante	23
7.4	Vies restantes, joueurs en vie, survivants	23
7.5	Comptabilité d'un résultat	23
7.6	Recalcul complet	24
7.7	Contrôle de cohérence	24
8	Moteur de propositions	25
8.1	Réparation d'un graphe désaccordé	25
8.2	Micro-rondes et pauses	25
8.2.1	Micro-rondes (<code>batch_minutes</code>)	25
8.2.2	Pauses (<code>Config.Breaks</code>)	26
8.3	Propose	26
8.4	Fin de phase	27
8.5	Matchs en cours	27
8.6	Attribution des tables	27
8.7	Générateur pseudo-aléatoire	27
8.8	Utilitaires d'ordre	28
9	Graphes de matchs	28
9.1	Accès	28
9.2	Résolution	28
9.3	Sélection des matchs lançables	29
9.4	Constructeurs de sections	29
9.4.1	Tableau à élimination simple	29
9.4.2	Consolante progressive	30
9.4.3	Tableau construit sur des sources (dernière chance)	31
9.4.4	Groupe GSL	31
9.4.5	Mini-tableau à élimination directe	31
9.4.6	Poule toutes rondes (table de Berger)	32
10	Format <code>swiss_lives</code> — suisse à vies	32
10.1	Principe	32
10.2	Bascule vers un tableau (<code>target</code>)	32
10.3	Joueurs libres et historique	33
10.4	Appariement d'un groupe	33
10.5	Proposition en mode continu	33
10.6	Proposition en mode par rondes	34
10.7	Classement d'une phase à vies	35
11	Format <code>gsl</code> — blocs de groupes	36
11.1	Principe	36
11.2	Partition d'une liste en groupes	36
11.3	Fin de bloc, fin de phase	37
11.4	Proposition	37
11.5	Application du tirage	37
12	Formats <code>bracket</code> et <code>lives_bracket</code> — tableaux	38
12.1	Utilitaire	38

12.2 Tirage des places	38
12.3 Construction du tableau	38
12.4 Application du tirage	39
12.5 Proposition	39
12.6 Classement d'un tableau	40
12.7 Survivants d'un tableau	41
13 Format round_robin — poules	41
13.1 Principe	41
13.2 Tirage des poules	41
13.3 Décompte et détection des égalités	41
13.4 Barrage	42
13.5 Proposition	42
13.6 Qualifiés et classement	43
14 Classement général et prix	43
14.1 Concaténation des phases	43
14.2 Prix	44
14.2.1 Pool, retenue, barème par section	44
14.3 Export CSV	45
15 Horloge et prévisions	45
16 Paquets annexes	46
16.1 <code>sim</code> — simulation et prévision	46
16.1.1 Modèle probabiliste	46
16.1.2 Exécution d'un tournoi	46
16.1.3 Prévision de fin	47
16.2 <code>render</code> — affichage	47
16.3 <code>players</code> — import CSV	48
16.4 Binaires de démonstration	49
16.4.1 <code>cmd/tournoi-demo</code>	49
16.4.2 <code>cmd/tournoi-td</code>	49
17 Invariants et plan de tests	50
17.1 Nature des tests	50
17.2 Invariants vérifiés	50
17.3 Fuzzing	50
17.4 Compatibilité ascendante	50
17.5 Invariants structurels à préserver	51
18 Limites connues et extensions prévues	51
18.1 Avant un premier tournoi réel	51
18.2 Fonctions attendues d'un logiciel de tournoi	51
18.3 Moteur : robustesse	52
19 Annexes	52
19.1 A. Récapitulatif de l'API publique	52
19.1.1 Types	52
19.1.2 Constantes	52
19.1.3 Fonctions et méthodes	52
19.2 B. Exemple de journal	53
19.3 C. Nomenclature des clés	54
19.4 D. Documents liés	54

1 Objet du document

Ce document spécifie **complètement** le moteur de tournoi de backgammon `tournoi`. Il est écrit pour être suffisant à lui seul : un développeur qui ne dispose que de ce texte doit pouvoir reconstruire une implémentation

qui, à partir du même journal d'événements et de la même graine, produit **les mêmes propositions, les mêmes états et les mêmes classements** que l'implémentation de référence.

Sont spécifiés :

- le modèle de données complet (types, champs, sérialisation JSON) ;
- le journal d'événements et la sémantique exacte de chaque événement ;
- la reconstruction de l'état (**Replay**, recalcul après correction) ;
- l'algorithme de proposition d'actions, format par format, au niveau du détail nécessaire à la reproduction bit à bit (ordres de tri, usage du générateur aléatoire) ;
- les graphes de matchs et leur résolution ;
- le classement, les prix, les exports ;
- les paquets annexes (simulation, rendu, import de joueurs) ;
- les invariants vérifiés et le plan de tests.

Ne sont **pas** spécifiés : l'interface utilisateur du logiciel hôte, la persistance, le réseau, l'authentification. Le moteur est une bibliothèque passive, sans état global, sans entrées-sorties.

1.1 Conventions de rédaction

- Le langage de référence est **Go 1.22**, sans dépendance externe (bibliothèque standard seule).
- Le code, les commentaires, les libellés produits et cette spécification sont **en français** ; seuls les identifiants exportés (noms de types et de fonctions) sont en anglais.
- Les noms de champs JSON sont donnés tels quels : ils font partie du contrat, puisque le journal est un document JSON persistant chez l'hôte.
- « TD » désigne le directeur de tournoi.
- Les extraits d'algorithme sont donnés en pseudo-code impératif ; quand l'ordre des opérations influence le résultat (tris, tirages), il est explicité.

1.2 Statut

Le moteur est fonctionnel et validé par simulation ; il n'a jamais dirigé de tournoi réel. Les limites connues et le travail restant sont listés au chapitre « Limites connues et extensions prévues », qui reprend docs/RESTE_A_FAIRE.md.

2 Principes de conception

Six principes gouvernent l'ensemble ; toute extension doit les respecter.

2.1 Journal d'événements, état dérivé

La **seule** donnée persistante est le journal : une liste ordonnée d'événements (**Journal**, tableau JSON). Le moteur ne persiste rien lui-même ; c'est l'hôte qui stocke le journal.

L'état complet du tournoi est une **fonction pure du journal** :

State = **Replay**(**Journal**)

Conséquences :

- une reprise après panne est un simple **Replay** ;
- un journal peut être rejoué à n'importe quel préfixe pour obtenir l'état à un instant passé (c'est ainsi que sont produites les pages « par étape ») ;
- deux exécutions du même journal donnent le même état, y compris les mêmes propositions.

2.2 On ne modifie jamais le journal

Une erreur du TD ne se corrige jamais en réécrivant ou en supprimant un événement, mais en **ajoutant** un événement de correction :

Erreur	Événement correctif
Mauvais vainqueur saisi	<code>result_corrected</code>
Match lancé par erreur	<code>match_cancelled</code>
Joueur qui abandonne	<code>player_withdrawn</code>

Ces trois événements déclenchent un **recalcul complet** de la comptabilité des phases à partir de la liste des matchs (`recompute`), suivi d'un contrôle de cohérence qui remplit `State.Warnings` (par exemple : « ce match de tableau a été joué par des joueurs que le tableau n'attendait pas »).

2.3 Le moteur propose, le TD dispose

Le moteur n'agit jamais de lui-même. Il expose :

```
Propose() []Action      // « voici ce qu'il y a à faire maintenant »
EventFromAction(a, now) // le TD confirme une action → un événement
Apply(ev)              // l'événement est appliqué à l'état
```

Chaque bouton de l'interface hôte produit **exactement un événement**. Le TD peut confirmer tout, partie, ou rien des actions proposées ; il peut aussi produire des événements que le moteur n'a pas proposés (forfait, correction, note).

2.4 Déterminisme du tirage au sort

`Propose` ne dépend que du journal. Le générateur pseudo-aléatoire utilisé pour une proposition est

```
rng = MT(Seed·1000003 + NEvents·7919 + Current)
```

où `Seed` est la graine du tournoi, `NEvents` le nombre d'événements déjà appliqués et `Current` l'indice de la phase en cours. Deux appels successifs à `Propose` sans événement intermédiaire renvoient donc **la même proposition**.

De plus, tout tirage réellement décidé (placement dans un tableau, composition des groupes) est **matérialisé dans l'événement draw** sous la forme d'une structure `Draw`. Rejouer un journal ne fait donc jamais appel à l'algorithme de tirage : on peut réécrire `drawSlots` ou `pairGroup` sans casser les journaux existants.

Enfin : **toute itération sur une table de hachage (map) dont l'ordre influencerait une proposition doit passer par un tri stable** (`sortedIDs`, tri par identifiant croissant). C'est une condition de correction, pas un détail de style : l'ordre d'itération des maps Go est délibérément aléatoire.

2.5 Aucun départage

C'est un choix de conception, justifié par l'étude des formats (`docs/etude_formats.md`) : les égalités ne sont **jamais** tranchées par un critère arithmétique (Buchholz, différence de points, départage au rating). Elles sont soit conservées comme **ex æquo** au classement, soit réglées par un **barrage** joué. Les formats retenus (vies, tableaux, poules) permettent toujours de conclure.

Corollaire : pas de têtes de série par défaut, pas de finale à handicap. Les têtes de série existent en **option** (`seeding`: "`rating`", voir plus bas) pour les organisateurs qui en veulent ; elles sont éteintes tant qu'on ne les demande pas.

2.6 Les graphes de matchs sont des données

Tableaux, groupes GSL, poules et consolantes sont tous décrits par la même structure : une `Section` contenant des `GMatch` dont chaque place est une `Src` — soit un joueur fixé, soit « le vainqueur du match i », soit « le perdant du match j de la section S ». Une fonction unique `resolve` propage les résultats jusqu'au point fixe.

Ajouter un format de tableau revient donc à écrire **un constructeur de section**, sans toucher au moteur.

3 Vocabulaire

Terme	Définition
Tournoi	Une configuration, une graine, un journal.
Phase	Une étape du tournoi, d'un type donné (suisse, tableau, GSL, poules). Les phases s'enchaînent linéairement.
Format	Le type d'une phase (<i>Kind</i>) : <code>swiss_lives</code> , <code>lives_bracket</code> , <code>gsl</code> , <code>bracket</code> , <code>round_robin</code> .
Vie	Droit à une défaite. Un joueur à 0 vie restante est éliminé de la phase.
Section	Un graphe de matchs nommé à l'intérieur d'une phase : <code>main</code> , <code>conso</code> , <code>last</code> , <code>gf</code> , un groupe GSL, une poule, un barrage.
GMatch	Une case du graphe : deux places, une longueur, un résultat éventuel. Pas encore un match joué.
Match	Un match réellement lancé : deux joueurs, une table, un horodatage. Relié à son GMatch par le couple (section, clé).
Bye / exemption	Place vide dans un tableau (BYE), ou tour sauté par un joueur en ronde synchrone.
Walkover	Victoire acquise sans jouer : contre une exemption, ou contre un joueur retiré.
Barrage	Mini-tournoi à 2 vies entre joueurs à égalité pour départager des places qualificatives.
Action	Proposition du moteur au TD.
Événement	Fait consigné au journal.
PR	<i>Performance Rating</i> du backgammon : plus bas = meilleur (2 = très fort, 10 = débutant).

4 Modèle de données

Tous les types de ce chapitre appartiennent au paquet `tournoi`.

4.1 Identifiants et joueurs

```
type PlayerID string           // fourni par l'hôte ; unique dans un tournoi
const BYE PlayerID = "BYE"    // place vide réservée dans un tableau
```

```
type Player struct {
    ID      PlayerID `json:"id"`
    Name    string   `json:"name"`
    Club    string   `json:"club,omitempty"`
    Rating  float64   `json:"rating,omitempty"` // PR ; 0 = inconnu
}
```

BYE est une valeur réservée : aucun joueur inscrit ne peut porter cet identifiant. Le moteur ne le vérifie pas ; c'est à l'hôte de l'interdire.

Le Rating n'est **jamais** utilisé par le moteur pour apparier ou classer. Il ne sert qu'à la simulation (`sim`) et à l'affichage. C'est une conséquence directe du principe « pas de têtes de série ».

4.2 Match

```
type MatchID string           // attribué par le moteur : "M1", "M2", ...
```

```
type MatchStatus string
const (
    Running    MatchStatus = "running"
```

```

    Finished MatchStatus = "finished"
    Cancelled MatchStatus = "cancelled"
)

type Match struct {
    ID          MatchID    `json:"id"`
    Phase       int         `json:"phase"`           // indice de la phase (0-based)
    Section     string      `json:"section,omitempty"` // "main", "conso", "B1G3"...
    Label       string      `json:"label,omitempty"`  // "Ronde 3", "Quart de finale"...
    Key         string      `json:"key,omitempty"`    // clé du GMatch dans sa section
    Round       int         `json:"round,omitempty"`  // ronde suisse (0 = sans objet)
    A           PlayerID    `json:"a"`
    B           PlayerID    `json:"b"`
    Length      int         `json:"length"`          // en points
    Table       int         `json:"table,omitempty"`
    Status      MatchStatus `json:"status"`
    Start       time.Time   `json:"start"`
    End         time.Time   `json:"end,omitempty"`
    Winner      PlayerID    `json:"winner,omitempty"`
    ScoreA      int         `json:"score_a,omitempty"`
    ScoreB      int         `json:"score_b,omitempty"`
    Forfeit     bool        `json:"forfeit,omitempty"`
}

```

Méthodes :

- `Loser()` `PlayerID` : renvoie B si `Winner == A`, sinon A. N'est valide que pour un match terminé ; sur un match non terminé elle renvoie A (puisque `Winner` est vide).
- `Has(p PlayerID) bool` : `A == p || B == p`.

Un match est relié à son emplacement dans un graphe par le couple (**Section**, **Key**). Les deux champs sont vides pour un match libre (suisse continu, barrage, finale GSL).

4.3 Rang

```

type Rank struct {
    Player PlayerID `json:"player"`
    Rank    int         `json:"rank"`           // 1 = premier ; les ex aequo partagent le rang
    Note    string      `json:"note,omitempty"` // "vainqueur", "finaliste", "3 victoires"...
}

```

Les notes sont aujourd'hui du français brut. Une implémentation qui vise l'internationalisation devrait leur substituer des codes ; le format des notes n'est pas contractuel.

4.4 Action

```

type ActionKind string
const (
    ActStartMatch ActionKind = "start_match" // lancer un match
    ActBye         ActionKind = "bye"         // donner un bye (ronde synchrone)
    ActDraw        ActionKind = "draw"        // effectuer un tirage (le tirage est joint)
    ActNextPhase   ActionKind = "next_phase"  // passer à la phase suivante
    ActCancelMatch ActionKind = "cancel_match" // annuler un match devenu incohérent
    ActFinish      ActionKind = "finish"      // clore le tournoi
    ActWait        ActionKind = "wait"        // rien à faire : attendre
)

type Action struct {
    Kind    ActionKind `json:"kind"`
    Phase   int         `json:"phase"`
    Section string      `json:"section,omitempty"`
    Label   Label       `json:"label,omitempty"`
}

```

```

Round    int           `json:"round,omitempty"`
Key       string        `json:"key,omitempty"`
Match    MatchID       `json:"match,omitempty"` // cancel_match : le match à annuler
A         PlayerID      `json:"a,omitempty"`
B         PlayerID      `json:"b,omitempty"`
Length    int           `json:"length,omitempty"`
Table     int           `json:"table,omitempty"`
Draw      *Draw         `json:"draw,omitempty"`
Reason    ReasonCode    `json:"reason,omitempty"` // ActWait : pourquoi
Until     time.Time     `json:"until,omitempty"` // waiting_batch : échéance du prochain lot
Warn      WarningCode   `json:"warn,omitempty"` // ce que le moteur remarque sans rien bloquer
}

```

Action.String() produit une phrase française destinée au TD :

Kind	Phrase
start_match	Lancer <Label> : <A> contre en <Length> points, table <Table>
bye	Bye pour <A> (<Label>)
draw	Tirage : <Label>
next_phase	Passer à la phase suivante : <Label>
cancel_match	Annuler <Match> : <A> contre (<Label>), devenu incohérent
finish	Clore le tournoi
(autre)	Attendre : <Reason>

4.5 Tirage

```

type Draw struct {
    Slots []PlayerID `json:"slots,omitempty"` // places du 1er tour (BYE = vide)
    Groups [][]PlayerID `json:"groups,omitempty"` // groupes (GSL, poules, barrage)
    Lives map[PlayerID]int `json:"lives,omitempty"` // vies à l'entrée (information)
}

```

Slots a toujours une longueur puissance de 2 2; la paire i du premier tour est (Slots[2i], Slots[2i+1]).

Lives est purement informatif (affichage, audit) : il n'est pas relu au jeu.

Le tirage étant **matérialisé** dans l'événement, l'algorithme qui le produit peut changer sans casser un journal existant. C'est ce qui a permis d'ajouter les têtes de série optionnelles (**seeding**) après coup : un journal écrit avant se rejoue place pour place.

4.6 État

```

type State struct {
    Config    Config           `json:"config"`
    Seed      int64           `json:"seed"`
    Players   map[PlayerID]*Player `json:"players"`
    Order     []PlayerID      `json:"order"` // ordre d'inscription
    Withdrawn map[PlayerID]bool `json:"withdrawn,omitempty"`
    Matches   map[MatchID]*Match `json:"matches"`
    MatchOrder []MatchID       `json:"match_order"` // ordre de lancement
    Phases    []*PhaseState    `json:"phases"` // uniquement les phases atteintes
    Current   int              `json:"current"` // phase en cours ; -1 = néant
    Finished  bool             `json:"finished"`
    Final     []Rank           `json:"final,omitempty"` // figé à la clôture
    NEvents   int              `json:"n_events"`
    Last      time.Time       `json:"last"` // horodatage du dernier événement
    Warnings  []Warning        `json:"warnings,omitempty"` // incohérences (codes)
    Infos     []Info           `json:"infos,omitempty"` // inscrits qui ne jouent nulle part
}

```

```

    Revived    map[PlayerID]bool    `json:"revived,omitempty"` // rendus à la vie par une correction,
    Unavailable map[PlayerID]Absence `json:"unavailable,omitempty"` // indisponibilités déclarées
    nextID     int                  // interne : compteur de matchs
}

```

Phases ne contient que les phases **atteintes** : à la création, une seule (Phases[0]); une phase est ajoutée à chaque next_phase. Config.Phasess contient la liste complète prévue.

```

type PhaseState struct {
    Index      int          `json:"index"`
    Cfg        PhaseConfig `json:"cfg"`
    Entrants   []PlayerID  `json:"entrants"` // ordre d'entrée dans la phase
    Lives      map[PlayerID]int `json:"lives"` // vies à l'entrée
    Losses     map[PlayerID]int `json:"losses"`
    Wins       map[PlayerID]int `json:"wins"`
    Byes       map[PlayerID]int `json:"byes"`
    Opponents  map[PlayerID][]PlayerID `json:"opponents"` // adversaires déjà rencontrés
    ElimOrder  []PlayerID  `json:"elim_order"` // ordre d'élimination
    Round      int         `json:"round"` // ronde synchrone / bloc GSL
    Roster     []PlayerID  `json:"roster,omitempty"` // suisse par rondes : appelés à l'ordre
    ByeRounds  map[PlayerID][]int `json:"bye_rounds,omitempty"` // rondes où le joueur a eu un bye
    Drawn      bool        `json:"drawn"` // le tirage a eu lieu
    Done       bool        `json:"done"`
    Sections   []*Section  `json:"sections,omitempty"`
    Length     int         `json:"length"` // longueur courante des matchs
    Started    bool        `json:"started"` // au moins un match lancé
}

```

Un PhaseState neuf est créé par newPhaseState(i, cfg) : toutes les maps initialisées vides, Length = cfg.Length, tout le reste à zéro.

4.7 Graphe de matchs

```

type Section struct {
    Name      string `json:"name"` // "main", "conso", "B2G3", "Poule A"...
    Kind      string `json:"kind"` // main/conso/last/gf/gsl/se/poule/barrage
    Group     int    `json:"group,omitempty"` // numéro de groupe/poule
    Block     int    `json:"block,omitempty"` // GSL : numéro de bloc
    Matches   []GMatch `json:"matches"`
    Rounds    [][]int `json:"rounds,omitempty"` // indices de Matches par tour
    Players   []PlayerID `json:"players,omitempty"` // barrage : joueurs concernés
    Spots     int    `json:"spots,omitempty"` // barrage : places à attribuer
}

```

```

type GMatch struct {
    Key      string `json:"key"` // unique dans la section
    Label    string `json:"label,omitempty"`
    Length   int    `json:"length"`
    Src      [2]Src `json:"src"` // origine des deux places
    Players  [2]PlayerID `json:"players"` // places résolues (" " = inconnue)
    MatchID  MatchID `json:"match_id,omitempty"` // match réel associé
    Winner   PlayerID `json:"winner,omitempty"`
    Loser    PlayerID `json:"loser,omitempty"`
    Done     bool    `json:"done"`
    Walkover bool    `json:"walkover,omitempty"` // gagné sans jouer (BYE, forfait)
    Skipped  bool    `json:"skipped,omitempty"` // conditionnel non joué (recharge)
    CondFrom int    `json:"cond_from,omitempty"`
    CondSide int    `json:"cond_side,omitempty"`
    Cond     bool    `json:"cond,omitempty"`
}

```

```

type Src struct {
    Player PlayerID `json:"player,omitempty"` // place fixée
    From    int      `json:"from"`           // sinon : indice du match source ; -1 = aucun
    Section string    `json:"section,omitempty"` // section source (" " = la même)
    Loser   bool      `json:"loser,omitempty"` // prendre le perdant au lieu du vainqueur
}

```

Règles :

- une **Src** est **soit** **Player** non vide (place fixée), **soit** **From** 0 (place héritée); **From** = -1 avec **Player** vide signifie « place définitivement vide », ce qui n'arrive pas dans les constructeurs actuels (les places vides sont représentées par **Player: BYE**);
- **Section** vide signifie « le match *From* de la section courante »;
- une section est **acyclique** : un **GMatch** ne référence que des matchs d'indice inférieur dans sa propre section, ou des matchs d'une section construite avant elle.

Match conditionnel (**Cond**) : le match n'existe que si le vainqueur du match **CondFrom** (même section) est le joueur qui occupait la place **CondSide** de ce match. Sert exclusivement à la recharge de la double élimination : la grande finale de repêchage n'a lieu que si le vainqueur de la consolante (place 1) a gagné la première grande finale.

4.8 Sérialisation

- **Journal.Bytes()** (**[]byte, error**) : **json.MarshalIndent(j, "", " ")** — un tableau JSON.
- **ParseJournal([]byte)** (**Journal, error**).
- **ParseConfig([]byte)** (***Config, error**) : déséréalise puis **valide** (voir chapitre suivant).

State est sérialisable en JSON (champs exportés) mais **n'est pas** un format d'échange : il est toujours reconstruit depuis le journal. Le champ privé **nextID** n'est pas sérialisé ; il est recalculé par le rejeu (incrémenté à chaque **match_started**).

5 Configuration

5.1 Structures

```

type Config struct {
    Name          string      `json:"name"`
    Phases        []PhaseConfig `json:"phases"`
    MinPerPoint   float64      `json:"min_per_point,omitempty"` // minutes/point ; défaut 8
    Tables        Tables       `json:"tables,omitempty"`        // tables de la salle
    Prizes        PrizePool    `json:"prizes,omitempty"`        // dotation (voir « Prix »)
    Breaks        []TimeRange  `json:"breaks,omitempty"`        // pauses programmées
}

```

```

type TimeRange struct { // Start include, End exclue
    Start time.Time `json:"start"`
    End    time.Time `json:"end"`
}

```

```

type PhaseConfig struct {
    Kind      string `json:"kind"`
    Name      string `json:"name,omitempty"`
    Lives     int    `json:"lives,omitempty"`
    Length    int    `json:"length"`
    FinalLength int    `json:"final_length,omitempty"`
    Lengths   []int  `json:"lengths,omitempty"`
    LengthLate int    `json:"length_late,omitempty"`
    LateThreshold int    `json:"late_threshold,omitempty"`
    Mode      string `json:"mode,omitempty"`
    Pairing   string `json:"pairing,omitempty"`
}

```

```

AvoidClubs      bool  `json:"avoid_clubs,omitempty"`
AllowRematch    bool  `json:"allow_rematch,omitempty"`
Target          int   `json:"target,omitempty"`
Consolation     bool  `json:"consolation,omitempty"`
LastChance      bool  `json:"last_chance,omitempty"`
Reconciliation  bool  `json:"reconciliation,omitempty"`
Recharge        bool  `json:"recharge,omitempty"`
GroupSize       int   `json:"group_size,omitempty"`
Qualifiers      int   `json:"qualifiers,omitempty"`
Entry          string `json:"entry,omitempty"`
}

```

5.2 Signification des champs de phase

Champ	Formats concernés	Signification
kind	tous	swiss_lives , lives_bracket , gsl , bracket , round_robin
name	tous	Nom affiché ; rempli par défaut si vide
lives	swiss_lives	Nombre de vies (défaut 2). Forcé à 2 pour gsl
length	tous	Longueur des matchs en points (obligatoire, > 0)
final_length	tableaux	Longueur du dernier tour (0 = length)
lengths	tableaux	Longueur tour par tour, du dernier tour vers le premier ([15,13,11,9] = finale 15, demies 13, quarts 11, reste 9). Plus précis que final_length et l'emporte sur lui ; une liste plus courte que le tableau retombe sur length
length_late	swiss_lives	Longueur des matchs de fin de phase, quand il reste au plus late_threshold joueurs en vie. Sans effet sans seuil (refusé par Validate) ; un length_changed du TD l'emporte
late_threshold	swiss_lives	Nombre de joueurs en vie à partir duquel length_late s'applique
mode	swiss_lives	continuous (défaut) ou rounds
batch_minutes	swiss_lives continu	Micro-rondes : les joueurs libres attendent l'échéance du prochain lot, puis tous ceux d'un même groupe de défaites sont appariés d'un coup. 0 = appariement au fil de l'eau. Refusé en mode rounds
pairing	swiss_lives	— une ronde EST un lot random (défaut) ou wins (appairer d'abord les joueurs les plus victorieux du groupe)
avoid_clubs	swiss_lives	Éviter les rencontres entre joueurs d'un même club quand c'est possible
allow_rematch	swiss_lives	Autoriser une seconde rencontre entre deux mêmes joueurs
target	swiss_lives , gsl	Figurer la phase quand Σ vies target (puissance de 2)

Champ	Formats concernés	Signification
consolation	bracket, lives_bracket	Consolante progressive alimentée par les perdants du principal
last_chance	idem	Dernière chance alimentée par les perdants de la consolante
reconciliation	idem	Le vainqueur de la consolante affronte celui du principal (grande finale)
recharge	idem	Double élimination vraie : le vainqueur du principal doit être battu deux fois
group_size	round_robin	Taille des poules (défaut 4)
qualifiers	round_robin	Qualifiés par poule (défaut 2)
entry	toutes sauf la première	survivors (défaut), all , top:N
seeding	bracket, lives_bracket	" (défaut) : tirage intégralement aléatoire. "rating" : placement classique par cote d'entrée (1 contre 16, 2 contre 15...). Le défaut vide est un choix de conception , pas un oubli : l'étude conclut « pas de têtes de série protégées », c'est la culture actuelle du backgammon. Refusé sur un format sans tirage de tableau

5.3 Validation et valeurs par défaut

`Config.Validate()` **error modifie la configuration en place** pour y écrire les défauts, et renvoie une erreur au premier problème. Elle est appelée par `New`, par `ParseConfig` et par `Apply(created)` : une configuration invalide ne peut donc pas entrer dans un journal.

Algorithme :

```

si len(Phases) == 0          → erreur « config : au moins une phase »
si MinPerPoint <= 0          → MinPerPoint = 8

pour chaque phase p d'indice i :
    si p.Length <= 0          → erreur « phase i : longueur de match manquante »

selon p.Kind :
    swiss_lives :
        si p.Lives <= 0          → p.Lives = 2
        si p.Mode == ""          → p.Mode = "continuous"
        si p.Mode {continuous, rounds} → erreur « mode inconnu »
        si p.Pairing == ""        → p.Pairing = "random"
        si p.Target != 0 et Target n'est pas une puissance de 2 → erreur
        si p.Target != 0 et p.Lives != 2 → erreur
        « la bascule vers un tableau à vies suppose 2 vies »
    lives_bracket, bracket :
        si p.Recharge et non p.Reconciliation → erreur « recharge sans reconciliation »
        si p.Reconciliation et non p.Consolation → erreur « reconciliation sans conso »
    gsl :
        p.Lives = 2 (forcé)
        si p.Target != 0 et Target n'est pas une puissance de 2 → erreur
    round_robin :
        si p.GroupSize <= 0 → p.GroupSize = 4
        si p.Qualifiers <= 0 → p.Qualifiers = 2
        si p.Qualifiers >= p.GroupSize → erreur « qualifiers doit être < group_size »
    défaut : erreur « type inconnu »

```

```

si p.Entry == "" → p.Entry = "survivors"
si p.Name == "" → p.Name = nomParDéfaut(p)

```

Le test de puissance de deux est `Target & (Target-1) == 0` (vrai aussi pour 0, d'où la garde `Target != 0`).

Noms par défaut :

Kind	Nom
swiss_lives	Suisse <Lives> vies
lives_bracket	Tableau final
gsl	Blocs GSL
bracket avec reconciliation	Double élimination
bracket sinon	Tableau
round_robin	Poules

Note d'implémentation importante : `Validate` étant idempotente et appliquée au rejeu, la configuration écrite dans l'événement `created` peut être partielle ; le rejeu produira la même configuration complétée. Une implémentation qui changerait les défauts changerait donc le sens des journaux anciens : les défauts font partie du contrat.

5.4 Configurations types

Les deux binaires de démonstration partagent le même catalogue nommé (il doit rester cohérent entre eux) :

Nom	Phases
suisse	swiss_lives 7 points
suisse_rondes	swiss_lives 7 points, mode: rounds
suisse_tableau	swiss_lives 7 pts target: 16, puis lives_bracket 9 pts, finale 11
gsl	gsl 7 pts target: 16, puis lives_bracket 9 pts
elim	bracket 9 pts, finale 11
conso	bracket 9 pts, consolation + last_chance
double	bracket 7 pts, consolation + reconciliation + recharge
poules	round_robin 5 pts (poules de 4, 2 qualifiés), puis bracket 9 pts

Format recommandé par l'étude : `suisse_tableau` — suisse 2 vies en continu, appariement aléatoire dans le groupe de même nombre de défaites, sans rematch, bascule à Σ vies = 16 ou 32, puis tableau à exemptions avec des matchs plus longs.

6 Journal et événements

6.1 Le catalogue des codes

Rien de ce qui sort du moteur n'est destiné à être affiché tel quel. Un logiciel hôte affiche un tournoi dans la langue de son utilisateur ; le moteur ne peut donc figer aucune langue, et surtout pas dans le journal, qui est conservé pour toujours.

Cinq familles de codes, toutes dans `codes.go` :

Famille	Type	Où elle apparaît
Libellés	Label{Kind, N, Losses, Match, Section, Text, Players, Spots, Sub}	Event.Label, Match.Label, Action.Label, GMatch.Label
Notes de classement	Note{Kind, Wins, Losses, Lives, Section, Qualified, Sub}	Rank.Note
Avertissements	Warning{Code, Match, Section, Label, A, B, ExpectedA, ExpectedB, Length, ScoreA, ScoreB, Player, Table, Other}	State.Warnings, Action.Warn (le code seul)
Informations	Info{Code, Player, Phase, Section, Label}	State.Infos
Raisons d'attente	ReasonCode	Action.Reason

Label.Sub compose un libellé dans un autre (« Bloc 2, groupe B : match décisif »). Label.Text est le **seul** champ texte admis, et il ne contient que ce que le TD a lui-même saisi — le nom d'une phase, la lettre d'une poule.

Les valeurs des codes entrent dans le journal : les changer casse les journaux existants. C'est la raison d'être de Event.Version.

Les **noms de section** (Section.Name, Match.Section, Event.Section) sont eux aussi des identifiants et non des libellés : main, conso, last, gf, poule:A, barrage:A, et les groupes GSL (B1G2). Ils voyagent dans le journal et dans la base de l'hôte.

Un test du dépôt (codes_test.go) refuse toute chaîne accentuée ou contenant un espace qui sortirait du moteur — propositions, classement, avertissements, informations, journal — hors les champs que le TD a saisis (text, name, club). Un autre vérifie que chaque code a un rendu français dans fr.go, et qu'aucun ne retombe sur sa valeur brute.

6.2 Structure d'un événement

```
type EventKind string
```

```
const (
    EvCreated      EventKind = "created"
    EvPlayerAdded  EventKind = "player_added"
    EvPlayerWithdrawn EventKind = "player_withdrawn"
    EvPlayerUpdated EventKind = "player_updated"
    EvPlayerUnavailable EventKind = "player_unavailable"
    EvPlayerAvailable EventKind = "player_available"
    EvMatchStarted EventKind = "match_started"
    EvResult       EventKind = "result"
    EvResultCorrected EventKind = "result_corrected"
    EvMatchCancelled EventKind = "match_cancelled"
    EvBye          EventKind = "bye"
    EvDraw         EventKind = "draw"
    EvNextPhase    EventKind = "next_phase"
    EvLengthChanged EventKind = "length_changed"
    EvConfigChanged EventKind = "config_changed"
    EvReopened     EventKind = "reopened"
    EvTableChanged EventKind = "table_changed"
    EvFinished     EventKind = "finished"
    EvNote         EventKind = "note"
)
```

```
type Event struct {
    Seq      int      `json:"seq"`
```

```

Version      int      `json:"version,omitempty"` // format du journal ; 0 = avant les codes
Kind         EventKind `json:"kind"`
Time         time.Time `json:"time"`
Config       *Config  `json:"config,omitempty"`
Seed         int64    `json:"seed,omitempty"`
Player       *Player  `json:"player,omitempty"`
ID           PlayerID `json:"player_id,omitempty"`
MatchID      MatchID  `json:"match_id,omitempty"`
Phase        int      `json:"phase,omitempty"`
Section      string    `json:"section,omitempty"`
Label        Label    `json:"label,omitempty"` // un CODE, jamais une phrase
Round        int      `json:"round,omitempty"`
Key          string    `json:"key,omitempty"`
A            PlayerID `json:"a,omitempty"`
B            PlayerID `json:"b,omitempty"`
Length       int      `json:"length,omitempty"`
Table        int      `json:"table,omitempty"`
Winner       PlayerID `json:"winner,omitempty"`
ScoreA       int      `json:"score_a,omitempty"`
ScoreB       int      `json:"score_b,omitempty"`
Forfeit      bool     `json:"forfeit,omitempty"`
AfterCurrent bool     `json:"after_current,omitempty"` // retrait différé
Draw         *Draw   `json:"draw,omitempty"`
Text         string  `json:"text,omitempty"`
Slot         string  `json:"slot,omitempty"` // retardataire : place d'exemption prise
Until        *time.Time `json:"until,omitempty"` // player_unavailable : heure de retour
}

```

type Journal []Event

Un événement est une structure « plate » : tous les types partagent les mêmes champs, les champs inutiles restant vides. C’est un choix délibéré — il rend le journal lisible et évite la désérialisation polymorphe.

Version est le format du journal (**JournalVersion**, actuellement 2). Un événement sans ce champ vient d’un journal antérieur aux codes structurés et est **converti à la lecture** (voir « Compatibilité ascendante » plus bas). Les constructeurs d’événements le posent : un **Event** écrit à la main sans version serait relu comme un journal ancien.

Label est un **code**, pas une chaîne. Voir « Le catalogue des codes » ci-dessous.

Seq est un numéro d’ordre facultatif renseigné par l’hôte (les binaires du dépôt écrivent **Seq = len(journal)** avant l’ajout). Le moteur **ne le lit pas** : l’ordre du tableau fait foi.

Time est l’horodatage. Le moteur ne l’utilise que pour :

- l’horodatage de début et de fin des matchs ;
- la mise à jour de **State.Last** (le maximum des horodatages vus).

Il n’exige pas la monotonie et ne s’en sert jamais pour décider.

6.3 Champs utilisés par type

Kind	Champs lus
created	Config (obligatoire), Seed
player_added	Player (obligatoire, ID non vide)
player_withdrawn	ID
player_updated	Player (obligatoire, ID d’un joueur inscrit)
player_unavailable	ID ; Until (heure de retour) ou Round (ronde de retour), ou aucun des deux
player_available	ID
match_started	MatchID, Phase, Section, Label, Key, A, B, Length, Table

Kind	Champs lus
result	MatchID, Winner, ScoreA, ScoreB, Forfeit
result_corrected	idem
match_cancelled	MatchID
bye	Phase, ID, Label
draw	Phase, Section, Draw (obligatoire)
next_phase	—
length_changed	Phase, Length
config_changed	Config (obligatoire, la configuration entière)
reopened	—
table_changed	MatchID, Table
finished	—
note	Text

6.4 Sémantique de Apply

func (s *State) Apply(ev Event) error applique un événement. **Première règle** : si `ev.Kind != EvCreated` et `s.Current < 0`, erreur « tournoi non créé ». Aucun événement ne peut précéder la création.

À la fin de tout traitement réussi, et pour **tous** les types y compris `note` :

```
s.NEvents++
si ev.Time > s.Last → s.Last = ev.Time
```

`NEvents` est donc le nombre d'événements appliqués avec succès ; il entre dans la graine du générateur, ce qui garantit qu'une nouvelle proposition suit toujours un tirage différent.

6.4.1 created

```
si ev.Config == nil          → erreur « created sans config »
cfg := *ev.Config ; cfg.Validate() → propage l'erreur éventuelle
s.Config, s.Seed = cfg, ev.Seed
s.Phases = [ newPhaseState(0, cfg.Phases[0]) ]
s.Current = 0
```

Un second `created` dans un journal écraserait la configuration ; le moteur ne l'interdit pas explicitement, mais l'hôte ne doit jamais le produire.

6.4.2 player_added

```
si ev.Player == nil ou ev.Player.ID == "" → erreur « joueur sans identifiant »
si l'identifiant est inconnu → l'ajouter à s.Order
s.Players[id] = copie de *ev.Player      (réinscription = mise à jour de la fiche)
supprimer id de s.Withdrawn              (une réinscription annule un forfait)

ph := phase courante
si ev.Slot != "" :
    takeSlot(phaseOf(ev.Phase), ev.Section, ev.Slot, id) ← erreur si la place n'est plus libre
sinon si ph.Index == 0 et non ph.Drawn et (ph.Kind == swiss_lives ou non ph.Started) :
    enter(ph, id, livesFor(ph.Cfg))
```

Retardataires. Trois chemins, et un seul interdit : refaire le tirage.

1. Le tirage n'a pas eu lieu (ou la phase est un suisse non figé) : le joueur entre tout de suite, avec toutes ses vies.
2. `ev.Slot` désigne une **place d'exemption libre** d'un tableau déjà tiré (`State.FreeSlots` les énumère, `PlayerAddedAtSlotEvent` construit l'événement) : il l'occupe là où elle est, avec une vie — il ne bénéficie pas de l'exemption qu'il prend. Une place dont le match a été lancé, ou dont le tour a commencé, ou qui n'existe pas, est **refusée** : `Apply` renvoie une erreur.
3. Sinon il est enregistré (`Players`, `Order`) sans entrer dans aucune phase, et `State.Infos` porte un code disant où il entrera (`enters_at`) ou que rien ne l'admet (`no_entry`).

6.4.3 player_updated

```
si ev.Player == nil ou ev.Player.ID == "" → erreur « joueur sans identifiant »
si l'identifiant est inconnu → erreur (une inscription est un player_added)
s.Players[id] = copie de *ev.Player
```

La **correction d'une fiche** — nom, club, cote — et rien d'autre. La fiche est entière, comme la configuration de `config_changed` : ce qui est écrit est le résultat. L'état du joueur n'est pas touché : un retiré reste retiré, ses vies, ses victoires, ses matchs et sa place dans les phases sont intacts.

Ce n'est **pas une réinscription**, et c'est ce qui la distingue de `player_added` sur un identifiant connu, qui met la fiche à jour **et** annule un retrait. Avant cet événement, corriger le club d'une joueuse retirée passait par `player_added` et la remettait en jeu ; un hôte devait réécrire le retrait juste après, et le journal portait deux événements pour un seul geste. `player_added` garde son sens pour les journaux existants — c'est aussi le chemin du retour d'un joueur retiré.

6.4.4 player_unavailable et player_available

```
si ID inconnu → erreur
player_available : supprimer s.Unavailable[ID]
player_unavailable :
  a := Absence{Until: *ev.Until (si présent), Round: ev.Round, Phase: s.Current}
  si Until et Round sont tous deux renseignés → erreur (une heure OU une ronde)
  si Round > 0 et la phase courante n'est pas un suisse par rondes → erreur
  s.Unavailable[ID] = a (remplace une absence précédente)
```

Un joueur **indisponible** n'est plus apparié, et ne perd rien : ni vie, ni victoire, ni rang, ni place dans un tableau. C'est le joueur qui ne revient que dimanche à 14 h, celui qui dîne, celui qui joue en ce moment dans une autre épreuve de la salle. Les trois chemins d'avant étaient faux : le retirer puis le réinscrire (« forfait » au classement pendant l'absence), ne rien faire (son appariement glisse, ou « tout lancer » le lance), ou un forfait (une vie perdue).

```
type Absence struct {
  Until time.Time `json:"until,omitempty"` // heure de retour ; zéro = pas d'échéance horaire
  Round int       `json:"round,omitempty"`  // suisse par rondes : ronde de retour
  Phase int       `json:"phase"`           // phase où elle a été déclarée
}
```

L'absence tombe :

- à l'heure `Until` (comparée à l'heure de la proposition, `ProposeAt(now)`, ou de l'événement en cours d'application) ;
- quand la ronde appariée atteint `Round`, ou au passage à une autre phase ;
- sans échéance, à `player_available`, qui lève aussi toute absence avant son terme.

Effets, tous dérivés :

- **suisse** : `free(ph, r)` écarte les absents — en continu comme par rondes (la ronde `r` est celle qu'on apparie). Un absent n'est pas dans le `Roster` de la ronde : la ronde se termine sans lui, **sans bye**. La file porte une attente par absent en vie (`wait`, raison `player_unavailable`, `A` = le joueur, `Until` ou `Round` = son retour) ;
- **graphes** (tableaux, GSL, poules, barrages) : la place est fixée, le match d'un absent reste donc proposé, mais **retenu** — raison `player_unavailable`, sans table. Le TD peut le lancer quand même.

6.4.5 player_withdrawn

```
si le joueur est inconnu → erreur
s.Withdrawn[id] = true
pour chaque match en cours où il joue :
  Status = Finished ; Forfeit = true ; End = ev.Time
  Winner = l'autre joueur
  onResult(match)
recompute()
```

Le forfait est **général** : le joueur quitte le tournoi. Ses matchs de graphe non encore lancés seront perdus par walkover lors de la résolution (voir **resolve**). Il n'existe pas aujourd'hui de forfait limité à un match, ni de retrait différé « à partir de la ronde suivante ».

6.4.6 match_started

Validations, dans l'ordre :

```

si MatchID vide ou déjà présent → erreur « identifiant invalide ou déjà utilisé »
si A == B, ou A vide, ou B vide → erreur « joueurs invalides »
pour p {A, B} :
    si p inconnu de s.Players → erreur « joueur inconnu »
    si busy(p) → erreur « a déjà un match en cours »

```

busy(p) : il existe un match de statut **Running** où p joue. **Un joueur ne peut jamais avoir deux matchs en cours.** C'est l'invariant central de l'ordonnancement.

Puis :

```

ph := phaseOf(ev.Phase) (si elle existe)
    openRound(ph, ev.Round) ← suisse par rondes, nouvelle ronde : relever ph.Roster
créer Match{ID, Phase, Section, Label, Key, Round, A, B, Length, Table,
    Status: Running, Start: ev.Time}
s.Matches[ID] = m ; s.MatchOrder += ID ; s.nextID++

ph (si elle existe) :
    ph.Started = true
    si ph.Kind == swiss_lives et ev.Round > ph.Round → ph.Round = ev.Round
    ph.Opponents[A] += B ; ph.Opponents[B] += A
    si un GMatch (Section, Key) existe → son MatchID = ID
check()

```

ev.Round (relu du libellé "Ronde %d" pour un journal de version 0, voir **Event.upgraded**) est le seul mécanisme qui fait avancer le compteur de rondes du suisse synchrone. **openRound** relève, au premier événement d'une ronde et avant de l'appliquer, les joueurs libres de la phase : ce sont les joueurs appelés à la ronde (voir « Proposition en mode par rondes »).

nextMatchID() renvoie "M" + (nextID+1). Comme **nextID** n'est incrémenté que par **match_started**, les identifiants sont consécutifs à partir de M1, sans trou, y compris après un rejeu partiel.

6.4.7 result et result_corrected

```

m := s.Matches[MatchID] ; si absent → erreur « match inconnu »
si Kind == result et m.Status == Finished
    → erreur « match déjà terminé (utiliser result_corrected) »
si Winner {m.A, m.B} → erreur « vainqueur absent du match »

```

```

m.Status = Finished ; m.Winner, m.ScoreA, m.ScoreB, m.Forfeit = ev...
si m.End est nul OU Kind == result → m.End = ev.Time

```

```

si Kind == result_corrected → avant := aliveInSwiss() ; recompute() ; noteRevived(avant)
sinon → onResult(m)

```

aliveInSwiss relève les joueurs en vie de chaque phase **swiss_lives**. **noteRevived** compare avant et après : un joueur que la correction rend à la vie entre dans **State.Revived**, un joueur qu'elle élimine en sort ; les avertissements sont recalculés (voir « Contrôle de cohérence »). **match_started** retire ses deux joueurs de **Revived**.

Une correction sur un match **annulé** le ramène à l'état **Finished** : c'est le moyen de revenir sur une annulation.

Noter que **result_corrected** sur un match jamais terminé fonctionne aussi : il vaut alors saisie initiale suivie d'un recalcul complet (plus coûteux, mais correct).

6.4.8 match_cancelled

```
m := s.Matches[MatchID] ; si absent → erreur
m.Status = Cancelled ; m.End = ev.Time
avant := aliveInSwiss() ; recompute() ; noteRevived(avant)
```

Le match reste dans `MatchOrder` et dans `Matches` : le journal ne perd rien. Il est simplement ignoré par tous les calculs (comptabilité, adversaires rencontrés, graphes).

6.4.9 bye

```
ph := phaseOf(ev.Phase) ; si absente → erreur « phase inconnue »
openRound(ph, ev.Round)
ph.Byes[ev.ID]++
si ev.Round > 0 → ph.ByeRounds[ev.ID] += ev.Round
si ph.Kind == swiss_lives et ev.Round > ph.Round → ph.Round = ev.Round
```

Un bye ne compte ni victoire ni défaite. Il **appartient à sa ronde** : le confirmer seul ne clôt pas la ronde, ses matchs restent proposés. Il est mémorisé pour éviter d'en donner deux au même joueur (voir `pairGroup`).

6.4.10 draw

```
ph := phaseOf(ev.Phase) ; si absente → erreur
si ev.Draw == nil → erreur « draw sans tirage »
applyDraw(ph, ev.Section, ev.Draw)
```

`applyDraw` dispatche selon le format (voir les chapitres de format) :

Kind	Traitement
bracket, lives_bracket	vérifie que <code>len(Slots)</code> est une puissance de 2, puis <code>buildBracket</code>
gsl	<code>applyGSLDraw</code> : nouveau bloc de groupes
round_robin	<code>applyRRDraw</code> : poules (Section vide) ou barrage (Section = "Barrage <poule>")
autre	erreur « la phase n'a pas de tirage »

6.4.11 next_phase

```
ph := phase courante
si ph == nil ou s.Current+1 >= len(Config.Phases) → erreur « pas de phase suivante »
ph.Done = true
next := newPhaseState(s.Current+1, Config.Phases[s.Current+1])
s.Phases += next ; s.Current++
enterFrom(next, ph)
```

6.4.12 length_changed

```
ph := phaseOf(ev.Phase) ; si absente → erreur
ph.Length = ev.Length
```

Change la longueur des **matchs futurs** de la phase. Les matchs déjà lancés gardent leur longueur ; les `GMatch` déjà construits (tableau tiré) gardent la leur aussi, puisqu'elle a été figée à la construction du graphe. Le champ n'a donc d'effet immédiat que sur les formats à appariement dynamique (suisse, barrages) et sur les graphes construits **après** le changement (blocs `GSL` suivants).

6.4.13 config_changed

```
si ev.Config == nil → erreur
cfg := copie profonde de *ev.Config ; cfg.Validate() → erreur éventuelle
acceptConfig(cfg, ev.Version) → erreur éventuelle (un *ConfigRefusal)
setConfig(cfg) ; recompute()
```

L'événement porte la configuration **entière**, et non le champ à changer : ce qui est écrit dans le journal est le résultat, pas l'instruction qui y mène — le même choix que pour **draw**. Un journal se relit alors sans connaître la règle de composition des retouches successives.

`acceptConfig` refuse trois choses, et seulement trois :

- une configuration qui a **moins de phases** que le tournoi n'en a ouvertes (une phase ouverte ne se retire pas) ;
- un changement de `Kind` sur une phase **terminée, tirée ou commencée** ;
- sur un tableau (`bracket`, `lives_bracket`) **déjà tiré**, un changement de l'une des options qui ne servent qu'à construire le graphe au tirage : `consolation`, `last_chance`, `reconciliation`, `recharge`, `seeding`. Le graphe est fait ; les changer ne pourrait plus rien produire. Avant ce refus, cocher la consolante d'un tableau tiré était accepté, écrit dans la configuration, et sans effet : le TD croyait avoir une consolante.

Le refus est un `*ConfigRefusal`, qui nomme la phase, le champ et la raison par des codes :

```
type ConfigRefusal struct {
    Phase int    // la phase qui bloque (removed : l'indice de la première phase retirée)
    Field string // champ JSON de PhaseConfig : "kind", "consolation"... ; "phases" pour un retrait
    Reason string // RefusalRemoved "removed", RefusalFinished "finished",
                // RefusalDrawn "drawn", RefusalStarted "started"
    From, To string // kind : l'ancien et le nouveau format
    Opened int      // removed : le nombre de phases déjà ouvertes
}
```

`Error()` en donne le rendu français (console, démo) ; un hôte multilingue traduit les codes.

Journaux anciens. Le troisième refus n'existe qu'à partir de la **version 2** du journal. Un `config_changed` de version 0 ou 1 qui changeait ces options après le tirage a été joué ainsi — accepté, sans effet — et se rejoue ainsi : `acceptConfig` reçoit la version de l'événement et n'applique le verrou du tirage qu'à partir de 2.

```
func (s *State) CheckConfig(next Config) error
```

`CheckConfig` est la **prévisualisation** : `Validate` puis `acceptConfig` sur une copie, aux règles de la version courante, sans rien appliquer. C'est ce qu'un formulaire de réglages appelle pour griser ou expliquer un champ avant d'écrire l'événement.

Tout le reste est accepté, y compris ce que le moteur ne peut pas juger : la bascule (**target**), les longueurs à venir, les tables, les pauses, la dotation, et une phase **ajoutée après** la phase courante.

`setConfig` répercute la nouvelle configuration sur les `PhaseState` déjà ouverts. La longueur courante d'une phase (`PhaseState.Length`) ne suit la configuration que si la configuration l'a **effectivement changée** : sinon, une retouche qui ne touche qu'à la bascule effacerait le `length_changed` que le TD venait de saisir à la main. Quand le `Kind` d'une phase non commencée change, les vies de ses entrants sont recalculées (`livesFor`).

6.4.14 reopened

```
si !s.Finished → erreur « le tournoi n'est pas clos »
s.Finished = false ; s.Final = nil
```

Rouvrir un tournoi clos, parce qu'un résultat était faux. Le classement final figé est effacé et sera recalculé à la clôture suivante ; le journal, lui, garde tout — la clôture, la réouverture, la correction et la nouvelle clôture sont quatre événements.

6.4.15 table_changed

```
m := s.Matches[MatchID] ; si absent → erreur « match inconnu »
m.Table = ev.Table
check()
```

Le déplacement est **toujours accepté**, y compris vers une table qui porte déjà un match en cours : le moteur avertit, il ne bloque pas. `check` lève alors `table_shared` (voir « Contrôle de cohérence »), qui tombe de lui-même dès que l'un des deux matchs se termine ou repart ailleurs. Refuser aurait laissé le TD sans recours dans la salle pleine où il en a justement besoin — un plateau cassé, quatorze tables sur quatorze occupées, deux joueurs qui finissent sur un coin de table en attendant.

6.4.16 finished

```
s.Finished = true
s.Final = s.Ranking()
```

Le classement est figé dans Final. StandingsCSV et les fonctions de rendu utilisent Final s'il existe, sinon recalculent Ranking().

6.4.17 note

Sans effet, sinon l'incrément de NEvents (ce qui décale le générateur aléatoire — une note change donc le prochain tirage : c'est sans conséquence sur la validité, mais à savoir).

6.5 Construction d'événements

```
func (s *State) EventFromAction(a Action, now time.Time) (Event, error)
func ResultEvent(id MatchID, winner PlayerID, scoreA, scoreB int, now time.Time) Event

// constructeurs d'événements (ils posent Version = JournalVersion ; ne jamais écrire un
// Event littéral sans version, il serait relu comme un journal ancien)
func PlayerAddedEvent(p Player, now time.Time) Event
func PlayerAddedAtSlotEvent(p Player, slot Slot, now time.Time) Event
func PlayerUpdatedEvent(p Player, now time.Time) Event // correction de fiche
func PlayerUnavailableEvent(id PlayerID, until time.Time, now time.Time) Event // until nul = jusqu'à
func PlayerUnavailableUntilRoundEvent(id PlayerID, round int, now time.Time) Event
func PlayerAvailableEvent(id PlayerID, now time.Time) Event
func PlayerWithdrawnEvent(id PlayerID, now time.Time) Event
func PlayerWithdrawnAfterCurrentEvent(id PlayerID, now time.Time) Event
func ForfeitEvent(id MatchID, winner PlayerID, now time.Time) Event
func CorrectionEvent(id MatchID, winner PlayerID, scoreA, scoreB int, now time.Time) Event
func CancelEvent(id MatchID, now time.Time) Event
func LengthChangedEvent(phase, length int, now time.Time) Event
func ConfigChangedEvent(cfg Config, now time.Time) Event
func ReopenedEvent(now time.Time) Event
func TableChangedEvent(id MatchID, table int, now time.Time) Event
func NoteEvent(text string, now time.Time) Event
func (e Event) WithNote(text string) Event
```

EventFromAction traduit une action confirmée en événement. Base commune : Event{Time: now, Phase: a.Phase, Section: a.Section, Label: a.Label, Key: a.Key}, puis :

Action	Événement produit
start_match	match_started avec MatchID = s.nextMatchID(), A, B, Length, Table
bye	bye avec ID = a.A
draw	draw avec Draw = a.Draw
cancel_match	match_cancelled avec MatchID = a.Match
next_phase	next_phase
finish	finished
autre (wait)	erreur « action sans événement associé »

Attention : l'identifiant de match est attribué au moment de l'appel. Si le TD confirme plusieurs actions start_match d'un même lot, il doit appeler EventFromAction puis Apply pour chacune, dans l'ordre, sans quoi deux matchs recevraient le même identifiant et le second Apply échouerait.

ResultEvent construit simplement {Kind: result, MatchID, Winner, ScoreA, ScoreB, Time}. Pour une correction, l'hôte réutilise la même structure en changeant Kind en result_corrected.

6.6 Boucle d'intégration côté hôte

```
cfg := tournoi.Config{Name: "Open", Phases: []tournoi.PhaseConfig{
    {Kind: tournoi.KindSwissLives, Length: 7, Target: 16},
    {Kind: tournoi.KindLivesBracket, Length: 9, FinalLength: 11},
}}
st, created, err := tournoi.New(cfg, seed, time.Now())
journal := tournoi.Journal{created}

// inscriptions
ev := tournoi.Event{Kind: tournoi.EvPlayerAdded, Time: time.Now(), Player: &p}
st.Apply(ev); journal = append(journal, ev)

// boucle du TD
for _, a := range st.Propose() { // « lancer P3 contre P8 en 7 points, table 4 »
    ev, _ := st.EventFromAction(a, time.Now()) // le TD confirme
    st.Apply(ev); journal = append(journal, ev)
}

// résultat saisi par le TD
ev = tournoi.ResultEvent("M12", "P3", 7, 4, time.Now())
st.Apply(ev); journal = append(journal, ev)

func (s *State) Step(evs ...Event) ([]Action, error) applique une suite d'événements puis renvoie
Propose(); c'est un raccourci, pas une primitive.
```

Règle d'or de l'hôte : un événement n'est ajouté au journal **que si Apply a réussi**. Un Apply en échec laisse l'état inchangé pour les validations en tête de traitement; pour les traitements qui modifient l'état avant de pouvoir échouer, l'hôte doit repartir d'un Replay.

7 Reconstruction de l'état

7.1 Replay et New

```
func Replay(j Journal) (*State, error)
func New(cfg Config, seed int64, now time.Time) (*State, Event, error)
```

Replay part d'un état neuf :

```
State{Players: {}, Withdrawn: {}, Matches: {}, Current: -1}
```

puis applique les événements dans l'ordre. À la première erreur, elle renvoie l'état partiel **et** une erreur enrichie : événement <i> (<kind>) : <erreur>.

New valide la configuration, construit l'événement `created` et l'applique; elle renvoie l'état, l'événement (que l'hôte doit ajouter en tête de journal) et l'erreur éventuelle.

7.2 Entrée dans une phase

```
func (s *State) enter(ph *PhaseState, p PlayerID, lives int)
```

Ajoute p à ph.Entrants (sans doublon) et fixe ph.Lives[p] = lives. Si p est déjà entrant, **rien n'est modifié** — y compris ses vies.

```
func livesFor(cfg PhaseConfig) int
```

Kind	Vies à l'entrée
swiss_lives	cfg.Lives
gsl	2
tout autre	1

7.3 Passage d'une phase à la suivante

```
func (s *State) enterFrom(next, prev *PhaseState)
selon next.Cfg.Entry :

    "all" :
        pour chaque p de s.Order dans l'ordre d'inscription, non retiré :
            enter(next, p, livesFor(next.Cfg))

    "top:N" :
        r := phaseRanking(prev)
        pour chaque rang rk de r (dans l'ordre du classement) :
            si rk.Rank <= N et p non retiré → enter(next, p, livesFor(next.Cfg))

    "survivors" (défaut) :
        pour chaque p de survivors(prev), non retiré :
            l := remainingLives(prev, p)
            si next.Kind {lives_bracket, gsl, swiss_lives} → l = livesFor(next.Cfg)
            enter(next, p, l)
```

C'est ici que se joue la **bascule à vies** : un joueur qui sort d'un suisse à 2 vies avec ses deux vies intactes entre dans un `lives_bracket` avec 2 vies, ce qui lui vaudra une **exemption du premier tour**. Un joueur qui n'a plus qu'une vie entre au premier tour.

`top:N` est lu par `Sscanf("%d")` ; un suffixe illisible donne `N = 0`, donc une phase vide. Le préfixe est reconnu par `len(Entry) > 4 && Entry[:4] == "top:"`.

7.4 Vies restantes, joueurs en vie, survivants

```
func (s *State) remainingLives(ph *PhaseState, p PlayerID) int
si p est retiré → 0
l := ph.Lives[p] - ph.Losses[p] ; si l < 0 → 0

func (s *State) alive(ph *PhaseState) []PlayerID
```

Les entrants (dans l'ordre de `Entrants`) dont `remainingLives > 0`.

```
func (s *State) survivors(ph *PhaseState) []PlayerID
```

Kind de la phase quittée	Survivants
<code>round_robin</code>	<code>rrQualified(ph)</code> — les qualifiés des poules, barrages inclus
<code>bracket, lives_bracket</code>	<code>bracketSurvivors(ph)</code> — le vainqueur si le tableau est fini, sinon les joueurs « en cours »
autres (<code>swiss_lives, gsl</code>)	<code>alive(ph)</code>

7.5 Comptabilité d'un résultat

```
func (s *State) onResult(m *Match)

ph := phaseOf(m.Phase) ; si absente → ne rien faire
loser := m.Loser()
ph.Wins[m.Winner]++
ph.Losses[loser]++
si remainingLives(ph, loser) == 0 et ph.Lives[loser] > 0 :
    ph.ElimOrder += loser
si un GMatch (m.Section, m.Key) existe :
    g.Done, g.Winner, g.Loser = true, m.Winner, loser
    ph.resolve(s.Withdrawn)
```

La garde `ph.Lives[loser] > 0` évite d'enregistrer comme « éliminé » un joueur qui n'était pas entrant de la phase (cas pathologique après correction).

ElimOrder sert au classement des phases à vies : le **dernier éliminé** quand il ne reste qu'un joueur en vie est le finaliste.

7.6 Recalcul complet

```
func (s *State) recompute()
```

Déclenché par `player_withdrawn`, `result_corrected` et `match_cancelled`.

1. Pour chaque phase :
 Losses, Wins, Opponents ← vides
 ElimOrder ← nil
 pour chaque GMatch de chaque section :
 Done, Winner, Loser, Walkover, Skipped ← false, "", "", false, false
 Players[k] ← Src[k].Player (donc "" pour une place dérivée d'un autre match)
2. Pour chaque match dans l'ordre de MatchOrder :
 ph := phaseOf(m.Phase) ; si absente ou m.Status == Cancelled → passer
 ph.Opponents[m.A] += m.B ; ph.Opponents[m.B] += m.A
 si un GMatch (m.Section, m.Key) existe → son MatchID = m.ID
 Puis, dans le même ordre : si m.Status == Finished → onResult(m)
3. Pour chaque phase : `resolve(s.Withdrawn)`
4. `s.Warnings = check()`

Ce qui **n'est pas** réinitialisé : `Byes` (un bye reste acquis), `Entrants`, `Lives`, `Round`, `Drawn`, `Sections` (structure) et `GMatch.MatchID`.

L'étape 2 fait **deux passes** : relier toutes les places à leur match, puis rejouer les résultats. En une seule passe, `resolve` — appelée par `onResult` — voyait une place encore sans match dont un joueur était retiré, et la déclarait gagnée par forfait (`Walkover`) alors que le match avait été joué avant le retrait ; le classement ignorait ce match, et une demi-finaliste retirée finissait dernière.

Les **places dérivées** (`GMatch.Players[k]` dont la source est un autre match) repartent vides à l'étape 1 et sont refaites par `resolve`. Sans cela, un match qui cesse d'être joué — correction, annulation, ou place d'exemption prise par un retardataire — laissait derrière lui le joueur qu'il avait fait avancer, et `resolve` ne le remplaçait jamais. Les places **fixes** (`Src[k].Player`, un joueur ou un `BYE`) sont restaurées telles quelles.

7.7 Contrôle de cohérence

```
func (s *State) check() []Warning
```

Pour chaque match non annulé rattaché à un `GMatch` dont les deux places sont connues : si l'ensemble {`m.A`, `m.B`} diffère de {`g.Players[0]`, `g.Players[1]`}, produire l'avertissement

`match <ID> (<section> <label>) : joueurs <A>/<B> mais le tableau attend <P0>/<P1>`

Pour chaque match terminé dont le score dépasse la longueur annoncée, l'avertissement `score_over_length`.

Dans les phases `swiss_lives`, deux avertissements disent ce qu'une correction (ou une annulation) a changé à la vie d'un joueur — un suisse n'a pas de graphe, donc pas de `bracket_wrong_players` :

- `correction_eliminatees_running` (`Match`, `Player`, `A`, `B`, `Label`) : un match **en cours** dont un joueur n'a plus aucune vie. Cela n'arrive pas dans un tournoi mené normalement — on ne lance pas un joueur éliminé — ; c'est la trace d'une correction d'un résultat antérieur qui l'a éliminé pendant qu'il jouait. L'annulation du match est proposée (voir « Réparation »). L'avertissement tombe quand le match finit ou est annulé.
- `correction_revives` (`Player`) : un joueur que la correction a rendu à la vie (`State.Revived`). Il est de nouveau appariable, et le moteur le dit jusqu'à ce qu'il rejoue — ou qu'une autre correction l'élimine de nouveau. Sans cela, la joueuse éliminée une heure à tort revenait dans les appariements sans que rien ne le signale.

Et pour chaque table qui porte **plusieurs matchs en cours**, l'avertissement `table_shared`, une fois par match arrivé en second dans `MatchOrder`, avec `Table`, `Match` (ce match) et `Other` (le premier occupant). Il

est dérivé de l'état : il disparaît quand l'un des deux matchs se termine, est annulé ou change de table. Un match sans table (`Table = 0`) ne partage rien. Le score est **enregistré** malgré tout : pendant un tournoi, c'est la parole du TD qui fait foi ; le moteur le signale, il ne le refuse pas.

`check` est rappelée après **tout événement qui touche un match** : `result`, `match_started` et `table_changed` directement, les autres (`result_corrected`, `match_cancelled`, `player_withdrawn`, `config_changed`) par `recompute`, qui la termine. Un avertissement qui n'apparaîtrait qu'après une correction sans rapport ne servirait à rien : le TD doit le voir au moment où il peut encore agir. `length_changed` ne touche que les matchs FUTURS d'une phase et ne change donc rien à ce que `check` regarde.

Elle parcourt les **graphes** et non les matchs — chaque place connaît son match par `MatchID` — parce que chercher, pour chaque match, sa place dans toutes les sections était quadratique : le coût était supportable une fois par correction, il ne l'est plus à chaque résultat. L'ordre des avertissements reste celui de `MatchOrder`, qui est celui que le TD lit.

`State.Warnings` doit être **vide** dans un tournoi mené normalement ; c'est un invariant vérifié par les tests. Sa non-vacuité signale au TD qu'une correction a désynchronisé un tableau, ou qu'un score est incohérent.

8 Moteur de propositions

8.1 Réparation d'un graphe désaccordé

Corriger un résultat de tableau après coup recalcule l'état et lève l'avertissement `bracket_wrong_players`, mais le directeur se retrouvait devant un arbre faux, avec les matchs suivants déjà joués par les mauvaises personnes, et rien pour l'aider.

`Propose` émet désormais la réparation comme des propositions ORDINAIRES, en tête de liste :

1. les places dont le match n'a pas été joué par les joueurs que la place attendait
2. + tout ce qui en descend (point fixe sur les `Src`, sections traversées comprises)
3. → une action `cancel_match` par place, de la plus profonde à la moins profonde

Un demi-finaliste faux fait une finale fausse, même si la finale oppose bien les deux joueurs que le graphe attendait : elle les attendait pour de mauvaises raisons. D'où l'annulation **en série**. Les annulations sont proposées du plus profond au moins profond — on défait le plus récent d'abord, comme sur le papier.

Les relances correctes ne sont pas fabriquées : une fois les annulations confirmées, les places redeviennent libres et sont proposées comme n'importe quel match prêt, à l'appel suivant. Tant que le TD n'a rien confirmé, il n'y a rien à relancer.

Rien n'est appliqué d'office. Ne rien confirmer laisse l'état tel quel, avertissement compris : un directeur peut préférer laisser le tableau tel qu'il a été joué et le noter à la main.

Dans un suisse, il n'y a pas de graphe à réparer, mais une correction peut éliminer un joueur en plein match (`correction_eliminate_running`). `proposeSwissRepair` propose alors, en tête de liste avec les autres réparations, un `cancel_match` du match en cours. Le TD peut aussi laisser finir le match : son adversaire perdra peut-être une vie contre un joueur éliminé, et c'est sa décision.

Corollaire dans `recompute` : `GMatch.MatchID` repart **vide** au recalcul, et n'est reposé que pour les matchs non annulés. Sans cela, une place dont le match vient d'être annulé gardait son identifiant, `ready` l'écartait, et le tableau restait bloqué à l'endroit même qu'on voulait rejouer.

8.2 Micro-rondes et pauses

Le moteur n'a pas d'horloge. Le temps entre par `ProposeAt(now)` — l'hôte donne la sienne — ou, à défaut, par l'horodatage du dernier événement du journal (`Propose() = ProposeAt(s.Last)`). Seul ce qui dépend du temps change avec `now` : les micro-rondes et les pauses. Le reste — appariements, tirages, passage de phase — ne dépend que du journal, et le rejeu n'est donc pas affecté : ce qui est rejoué, ce sont les événements, pas les propositions.

8.2.1 Micro-rondes (`batch_minutes`)

L'appariement au fil de l'eau rend le suisse continu manipulable par l'heure d'annonce d'un résultat : celui qui finit à 14 h 03 choisit son adversaire en annonçant à 14 h 04 ou à 14 h 20.

```

échéance := (départ du dernier match lancé dans la phase) + batch_minutes
si aucun match lancé      → pas d'échéance : le premier lot part tout de suite
si now < échéance          → une seule action :
    {wait, reason: waiting_batch, until: échéance}
sinon                     → appariement ordinaire du suisse

```

L'horodatage du dernier lot **n'est stocké nulle part** : c'est le départ du dernier match lancé dans la phase, un lot étant exactement cela — un paquet de matchs lancés ensemble. Le déduire évite un champ d'état de plus à tenir juste après une correction ou une annulation.

Action.Until porte l'échéance pour que l'hôte affiche un compte à rebours.

8.2.2 Pauses (Config.Breaks)

```

pour chaque action start_match :
    fin := now + Expected(Length)
    si [now, fin] rencontre une pause → Action.Warn = ends_in_break

```

Rien n'est bloqué. Le directeur sait des choses que le moteur ignore — que ces deux-là mangeront après, que la pause est indicative, que la salle ferme de toute façon. On le lui dit, il décide ; c'est la règle de tous les avertissements du moteur.

L'avertissement tombe dès que le match **rencontre** la pause, et pas seulement s'il s'y termine : un match lancé à midi moins dix et attendu pour 13 h 30 fait manquer le repas tout autant.

8.3 Propose

```

func (s *State) Propose() []Action // = ProposeAt(s.Last)
func (s *State) ProposeAt(now time.Time) []Action // = ProposeWith(now, External{})
func (s *State) ProposeWith(now time.Time, ext External) []Action

type External struct {
    BusyTables []int `json:"busy_tables,omitempty"` // tables occupées hors de ce tournoi
    BusyPlayers []PlayerID `json:"busy_players,omitempty"` // joueurs de ce tournoi qui jouent ailleurs
}

```

`External` est ce que l'hôte sait de la salle et que le journal de ce tournoi ignore : les tables où joue, en ce moment, **une autre épreuve** dirigée dans la même salle. Ce n'est pas un événement : l'occupation par l'extérieur change à chaque match de l'autre épreuve, elle n'appartient pas à l'histoire de celle-ci, et le rejeu n'en a pas besoin — le `match_started` confirmé porte la table effectivement prise. Seule l'attribution des tables en dépend.

`BusyPlayers` dit quels joueurs de **ce** tournoi jouent en ce moment dans une autre épreuve de la salle — un joueur inscrit à deux épreuves y a deux `PlayerID`, et c'est l'hôte qui les relie. Un identifiant inconnu est ignoré. Ces joueurs sont écartés des appariements dynamiques (suisse, barrages de poule) et la file porte, au suisse, une attente par joueur en vie (`wait`, raison `player_busy`, `A` = le joueur) ; dans un graphe, leur match reste proposé mais **retenu** — raison `player_busy`, sans table. Une indisponibilité (`player_unavailable`), écrite au journal, l'emporte dans la raison. Rien n'est écrit : le TD qui lance quand même le match à la main est obéi, et le rejeu l'ignore.

C'est le point d'entrée principal du moteur. Il est **pur** : il ne modifie pas l'état (à l'exception de l'écriture des numéros de table dans les actions renvoyées) et ne dépend que du journal déjà appliqué.

```

si s.Current < 0 ou s.Finished → nil

ph := phase courante
acts := selon ph.Cfg.Kind :
    swiss_lives      → proposeSwiss(ph)
    gsl              → proposeGSL(ph)
    lives_bracket, bracket → proposeBracket(ph)
    round_robin      → proposeRR(ph)

si acts est vide :
    si runningInPhase(ph) > 0 :

```

```

    → [ Wait, raison « matchs en cours » ]
si phaseDone(ph) :
    si une phase suivante existe → [ NextPhase, Label = nom de la phase suivante ]
    sinon                          → [ Finish ]
→ [ Wait, raison « aucun appariement possible » ]

si la phase n'est pas un suisse : holdAbsent(acts) ← match d'un absent : raison player_unavailable
                                holdElsewhere(acts) ← joueur dans ext.BusyPlayers : raison player_busy
assignTables(acts, ext.BusyTables) ← une action retenue n'a pas de table
→ acts + une attente player_unavailable par absent en vie (suisse)
    + une attente player_busy par joueur en vie occupé ailleurs, ni absent ni en match ici

```

L'ordre du repli est important : **attendre la fin des matchs en cours a priorité sur la clôture de la phase**. Le cas Wait / aucun appariement possible ne devrait pas se produire dans un tournoi sain ; il signale une impasse (par exemple deux joueurs libres qui se sont déjà rencontrés et `allow_rematch` désactivé — situation que les replis de `proposeSwiss` évitent).

8.4 Fin de phase

```
func (s *State) phaseDone(ph *PhaseState) bool
```

Kind	Condition
swiss_lives	swissDone(ph)
gsl	gslDone(ph)
round_robin	ph.Drawn && ph.allDone() && rrBarragesDone(ph)
bracket, lives_bracket	ph.Drawn && ph.allDone()

`ph.allDone()` : tous les `GMatch` de toutes les sections sont Done (un match sauté compte comme fait). Une phase de tableau non encore tirée n'est jamais « faite ».

8.5 Matchs en cours

```

func (s *State) Running() []*Match // tous les matchs de statut Running, dans MatchOrder
func (s *State) runningInPhase(ph) int // leur nombre dans une phase donnée
func (s *State) busy(p PlayerID) bool // p a-t-il un match en cours ?

```

8.6 Attribution des tables

```

func (s *State) assignTables(acts []Action, dehors []int)

used := { table de chaque match en cours, si > 0 } { tables de dehors }
pour chaque action du lot, dans l'ordre :
    si ce n'est pas un start_match, ou si Table est déjà renseignée → passer
    t := la plus petite table t - 1 telle que
        non used[t] et Config.Tables.AvailableFor(t, section, phase)
    si aucune (t > Tables.Count quand Count > 0) :
        Action.Reason = waiting_table ; passer
    Action.Table = t ; used[t] = true

```

`AvailableFor` écarte les tables hors service (`Tables.Unavailable`) et celles qui sont réservées à une autre section ou à une autre phase (`Tables.Reserved`). `dehors` est `External.BusyTables` : les tables où joue une autre épreuve de la salle. La table proposée est donc toujours **la plus petite libre**. Une action qui n'en trouve pas garde `Table = 0` et porte la raison `waiting_table` : elle reste confirmable (le moteur ne bloque pas), le TD saisit une table ou attend qu'une se libère.

8.7 Générateur pseudo-aléatoire

```

func (s *State) rng() *rand.Rand {
    return rand.New(rand.NewSource(
        s.Seed*1000003 + int64(s.NEvents)*7919 + int64(s.Current)))
}

```

Le générateur est **recréé à chaque appel** et dépend uniquement de la graine, du nombre d'événements et de la phase. Le suisse par rondes a le sien, `roundRng`, qui ne dépend que de la graine, de la phase et du numéro de ronde (voir « Proposition en mode par rondes »). Toute implémentation visant la compatibilité binaire avec la référence doit reproduire le générateur `math/rand` de Go (source *Additive Lagged Fibonacci* de la bibliothèque standard) et sa méthode `Shuffle`. **Une implémentation dans un autre langage ne pourra pas reproduire les propositions à l'identique** ; elle reproduira en revanche exactement les états et les classements à partir d'un journal donné, puisque les tirages y sont matérialisés. C'est la propriété qui compte.

8.8 Utilitaires d'ordre

```
func sortedIDs(ids []PlayerID) []PlayerID
```

Renvoie une **copie** triée par ordre croissant de l'identifiant (comparaison de chaînes). C'est le préalable obligatoire à tout mélange aléatoire ou tri partiel : sans lui, l'ordre d'entrée dépendrait de l'itération d'une map, et les propositions ne seraient plus déterministes.

9 Graphes de matchs

Ce chapitre spécifie `graph.go`, commun à tous les formats à graphe.

9.1 Accès

```
func (ph *PhaseState) gmatch(section, key string) *GMatch // nil si introuvable
func (ph *PhaseState) section(name string) *Section       // nil si introuvable
```

9.2 Résolution

```
func (ph *PhaseState) resolve(withdrawn map[PlayerID]bool)
```

Fonction centrale : elle propage les résultats dans tous les graphes de la phase **jusqu'au point fixe**. Elle est idempotente et peut être appelée aussi souvent qu'on veut.

répéter tant que quelque chose a changé :

pour chaque section, pour chaque GMatch g de la section :

- (a) match conditionnel
 - si g.Cond :
 - from := section.Matches[g.CondFrom]
 - si from.Done et non g.Skipped et non g.Done :
 - si from.Winner != from.Players[g.CondSide] :
 - g.Skipped = true ; g.Done = true ; changé ; passer au GMatch suivant
- (b) propagation des places, pour k = 0 puis 1 :
 - src := g.Src[k]
 - p := ""
 - si src.Player != "" → p = src.Player
 - sinon si src.From >= 0 :
 - sec := (src.Section == "") ? la section courante : ph.section(src.Section)
 - si sec existe, src.From est un indice valide, et sec.Matches[src.From].Done :
 - from := sec.Matches[src.From]
 - si from.Skipped → passer (place indéterminée)
 - p := from.Loser si src.Loser, sinon from.Winner
 - si p != "" et g.Players[k] != p → g.Players[k] = p ; changé
- (c) forfait : match non joué contre un joueur retiré
 - si non g.Done, g.MatchID == "", les deux places connues et différentes de BYE,
 - et au moins l'un des deux joueurs est retiré :
 - g.Done = true ; g.Walkover = true
 - g.Winner, g.Loser = Players[0], Players[1]

si `Players[0]` est retiré et `Players[1]` ne l'est pas → inverser
changé ; passer au `GMatch` suivant

(d) exemptions

si non `g.Done` et les deux places connues :
si les deux sont `BYE` → `Done`, `Walkover`, `Winner = BYE`, `Loser = BYE`
sinon si l'une est `BYE` → `Done`, `Walkover`, le joueur réel gagne, `Loser = BYE`

Points d'attention :

- l'ordre (a) (b) (c) (d) est significatif;
- (c) ne s'applique **pas** si le match a déjà été lancé (`MatchID != ""`) : un match commencé se conclut par un résultat de forfait, pas par un walkover de graphe;
- si les deux joueurs sont retirés, l'ordre du tableau tranche (`Players[0]` gagne) ; le vainqueur fictif sera lui-même éliminé par walkover au tour suivant ;
- la propagation d'un `BYE` gagnant fait remonter `BYE` comme vainqueur d'une paire entièrement vide, ce qui vide toute une branche du tableau — comportement voulu.

9.3 Sélection des matchs lançables

func (ph *PhaseState) ready() []readyMatch

Tous les `GMatch`, dans l'ordre des sections puis des matchs, tels que : non `Done`, sans `MatchID`, deux places connues, aucune place à `BYE`.

func (s *State) readyFree(ph *PhaseState) []readyMatch

Filtre `ready()` : on écarte tout match dont un joueur est déjà occupé (**busy**) **ou déjà retenu par un match précédent du même lot**. Le résultat est donc un lot de matchs simultanément lançables, sans conflit de joueur.

func (ph *PhaseState) allDone() bool

Tous les `GMatch` de toutes les sections sont `Done`.

9.4 Constructeurs de sections

Un constructeur ne fait que **poser des Src** ; il ne consulte jamais l'état. Ajouter un format de tableau se réduit à écrire un constructeur.

9.4.1 Tableau à élimination simple

func bracketSection(name, kind string, slots []PlayerID, lp lengthPlan) *Section

La longueur des matchs vient d'un **plan** qui superpose trois sources, de la plus précise à la plus générale — la liste par tour, la longueur de finale, la longueur par défaut :

```
type lengthPlan struct {
    def      int    // longueur par défaut de la phase (ph.Length)
    final    int    // Cfg.FinalLength ; 0 = def
    byRound []int  // Cfg.Lengths, du DERNIER tour vers le premier
}
```

```
lp.at(r, rounds) :
    i := rounds - 1 - r
    si 0 <= i < len(byRound) et byRound[i] > 0 → byRound[i]
    si r == rounds-1 et final > 0                → final
    sinon                                          → def
```

`plain(l)` est le plan d'une longueur unique (mini-tableaux, dernière chance) ; `bracketLengths(ph)` celui d'un tableau de la phase.

`slots` a une longueur `size` puissance de 2. `rounds = log2(size)`.

Tour 0 : pour `i` de 0 à `size/2 - 1`
Key = "<name>.0.<i>"

```

Label = roundLabel(0, rounds)
Length = lp.at(0, rounds)
Src = { {Player: slots[2i], From: -1},
        {Player: slots[2i+1], From: -1} }
Rounds[0] = les indices de ces matchs

Tour r, de 1 à rounds-1 : pour i de 0 à len(Rounds[r-1])/2 - 1
    Key = "<name>.<r>.<i>"
    Label = roundLabel(r, rounds)
    Length = lp.at(r, rounds)
    Src = { {From: Rounds[r-1][2i]}, {From: Rounds[r-1][2i+1]} }

```

Libellés de tour :

```
func roundLabel(r, rounds int) string
```

rounds - r	Libellé
1	Finale
2	Demi-finale
3	Quart de finale
autre	Tour <r+1>

Le dernier match de la section (Matches[len-1]) est toujours la finale : c'est lui qui désigne le vainqueur de la section.

9.4.2 Consolante progressive

```
func consolationSection(name string, main *Section, length int) *Section
```

La consolante recueille les perdants du tableau principal au fur et à mesure. Sa structure alterne un **tour de fusion** (survivants de la consolante contre les nouveaux perdants du principal) et un **tour interne** (survivants entre eux).

```

rounds := len(main.Rounds)
si rounds < 2 → section vide

Tour 0 (« Consolante tour 1 ») : pour i = 0, 2, 4, ... sur main.Rounds[0]
    Src = { perdant de main.Rounds[0][i], perdant de main.Rounds[0][i+1] }
si len(main.Rounds[0]) == 1 → renvoyer la section (tableau de 2 : pas de consolante)
Rounds[0] = ces matchs ; prev = ces matchs ; cr = 1

pour r de 1 à rounds-1 :
    drops := main.Rounds[r]
    fusion : pour i de 0 à len(prev)-1
        d := drops[len(drops)-1-i] ← ordre INVERSÉ
        Label = "Consolante tour <cr+1>"
        Src = { vainqueur de prev[i], perdant de d (section main) }
    Rounds += ces matchs ; prev = ces matchs ; cr++
    si len(prev) == 1 → sortir

    tour interne : pour i = 0, 2, 4, ... sur prev
        Label = "Consolante tour <cr+1>"
        Src = { vainqueur de prev[i], vainqueur de prev[i+1] }
    Rounds += ces matchs ; prev = ces matchs ; cr++

```

le dernier match construit reçoit le libellé « Finale consolante »

L'**ordre inversé** des drops (drops[len-1-i]) sert à éviter qu'un joueur retombant du principal ne retrouve immédiatement l'adversaire qui vient de le battre, ou un joueur de la même moitié de tableau. Il ne garantit rien au-delà du tour immédiat ; c'est un point ouvert (mesurer la fréquence des rematches par simulation, permuter les drops si nécessaire — le tirage étant enregistré, l'algorithme peut évoluer sans casser les journaux).

9.4.3 Tableau construit sur des sources (dernière chance)

```
func bracketFromSrcs(name, kind string, srcs []Src, length int) *Section
size := pow2ceil(len(srcs)) ; au minimum 2
construire un bracketSection de `size` places toutes à BYE
remplacer, dans l'ordre des matchs du premier tour :
    d'abord la place 0 de chaque match, avec srcs[0], srcs[1], ...
    puis la place 1 de chaque match, avec les sources restantes
libellés : matchs du premier tour → « Dernière chance, tour 1 »
          autres                  → « Dernière chance, » + libellé de tour
```

Remplir toutes les places 0 avant les places 1 **répartit les entrants** : avec 5 sources sur 8 places, on obtient trois joueurs exemptés plutôt qu'un déséquilibre.

9.4.4 Groupe GSL

```
func gslSection(name string, players []PlayerID, length int) *Section
```

Le format GSL (groupe de 4 à double élimination, 5 matchs) donne deux qualifiés : le vainqueur du match des gagnants (0 défaite) et le vainqueur du match décisif (1 défaite).

Groupe de 4 joueurs a, b, c, d — clés <name>.0 à <name>.4 :

#	Libellé	Places
0	Ouverture	a — b
1	Ouverture	c — d
2	Match des gagnants	vainqueur 0 — vainqueur 1
3	Match des perdants	perdant 0 — perdant 1
4	Match décisif	perdant 2 — vainqueur 3

Rounds = [[0,1], [2,3], [4]].

Groupe de 3 joueurs a, b, c :

#	Libellé	Places
0	Ouverture	a — b
1	Match des gagnants	vainqueur 0 — c
2	Match décisif	perdant 0 — perdant 1

Rounds = [[0], [1], [2]].

Groupe de 2 : un seul match, libellé Match, Rounds = [[0]].

Groupe de taille autre : section vide (le cas est écarté en amont).

9.4.5 Mini-tableau à élimination directe

```
func seSection(name string, players []PlayerID, length int) *Section
```

Pour un groupe de 2 à 4 joueurs à une vie (blocs GSL des joueurs déjà battus une fois) :

```
4 joueurs → slots = [p0, p1, p2, p3]
3 joueurs → slots = [p0, p1, p2, BYE]
2 joueurs → slots = [p0, p1]
autre      → section vide
sec := bracketSection(name, "se", slots, plain(length))
tous les libellés ← « Élimination directe »
```

9.4.6 Poule toutes rondes (table de Berger)

```
func rrSection(name string, players []PlayerID, length int) *Section
ids := players ; n := len(ids)
si n est impair → ajouter BYE ; n++

pour r de 0 à n-2 :
    pour i de 0 à n/2 - 1 :
        a, b := ids[i], ids[n-1-i]
        si a == BYE ou b == BYE → passer (pas de match)
        Key   = "<name>.<r>.<i>"
        Label = "Poule, ronde <r+1>"
        Src   = { {Player: a}, {Player: b} }
        Rounds += les indices produits
    rotation : ids := [ids[0], ids[n-1], ids[1], ids[2], ..., ids[n-2]]
```

Le premier élément reste fixe, les autres tournent d'un cran : c'est la table de Berger classique. Chaque joueur rencontre chaque autre exactement une fois, en $n-1$ rondes.

Les places d'une poule sont **toutes fixées** (`Src.Player`) : les matchs sont donc tous lançables dès le tirage, sous la seule contrainte qu'un joueur ne joue qu'un match à la fois. C'est ce qui donne aux poules leur souplesse d'ordonnement.

10 Format `swiss_lives` — suisse à vies

10.1 Principe

Un suisse à L vies est une **double élimination sans tableau** (pour $L = 2$) : chaque joueur est éliminé après L défaites, et l'appariement se fait toujours **à l'intérieur du groupe des joueurs ayant le même nombre de défaites**. Tout effectif est admis, la fin est toujours résoluble sans départage, et il n'y a pas de graphe à construire.

Deux modes :

- **continuous** (défaut) : dès que deux joueurs du même groupe sont libres, un match est proposé. C'est le mode recommandé — l'étude montre que l'attente est le premier gaspillage d'un tournoi (10 à 15 fois plus d'attente en synchrone qu'en continu).
- **rounds** : tous les matchs d'une ronde sont proposés ensemble, byes compris, et la ronde suivante n'est proposée qu'une fois tous les matchs terminés.

10.2 Bascule vers un tableau (`target`)

```
func (s *State) sumLives(ph *PhaseState) int    //  $\Sigma$  remainingLives sur les entrants
func (s *State) swissFrozen(ph *PhaseState) bool
func (s *State) swissDone(ph *PhaseState) bool
```

`swissFrozen` : `Target > 0` et `sumLives - runningInPhase <= Target`

```
swissDone :
    si runningInPhase > 0                → faux
    si Target > 0 et sumLives <= Target  → vrai
    → len(alive) <= 1
```

L'idée : chaque match consomme exactement une vie. Quand la somme des vies restantes atteint une puissance de 2, on peut basculer sur un tableau à exemptions parfaitement équilibré — un joueur à 2 vies occupe deux places du tableau (il est exempt du premier tour), un joueur à 1 vie en occupe une. C'est pourquoi `Validate` impose `Lives == 2` avec `Target`.

Sans `Target`, la phase va jusqu'à son terme naturel : un seul joueur en vie.

10.3 Joueurs libres et historique

```
func (s *State) free(ph *PhaseState) []PlayerID // alive(ph) privé des joueurs occupés
func (s *State) met(ph *PhaseState, a, b PlayerID) bool // b figure dans ph.Opponents[a]
func (s *State) sameClub(a, b PlayerID) bool
```

sameClub : les deux joueurs sont connus, leur club est non vide et identique.

10.4 Appariement d'un groupe

```
func (s *State) pairGroup(ph *PhaseState, g []PlayerID, rng *rand.Rand,
                        allowRematch bool) (pairs [][]PlayerID, rest []PlayerID)
```

C'est le cœur de l'appariement suisse. Glouton, sur une liste dont l'ordre a été construit en plusieurs passes :

```
1. g := sortedIDs(g)                ← ordre déterministe de départ
2. rng.Shuffle(g)                    ← mélange aléatoire
3. si Cfg.Pairing == "wins" :
    tri stable décroissant sur ph.Wins ← les plus victorieux en tête
4. tri stable décroissant sur ph.Byes ← ceux qui ont déjà eu un bye d'abord

5. libres := g
   tant que len(libres) >= 2 :
       a := libres[0] ; retirer a de libres
       passes := [faux]                ← une seule passe
       si Cfg.AvoidClubs → passes := [vrai, faux] ← d'abord en évitant les clubs
       trouvé := -1
       pour exigerClub dans passes :
           pour chaque c de libres, dans l'ordre :
               si non allowRematch et non Cfg.AllowRematch et met(a, c) → passer
               si exigerClub et sameClub(a, c) → passer
               trouvé := index de c ; sortir
           si trouvé >= 0 → sortir
       si trouvé < 0 : rest += a ; continuer
       pairs += (a, libres[trouvé]) ; retirer libres[trouvé]

6. rest += ce qui reste dans libres ← au plus un joueur en fin de boucle
```

Propriétés :

- le tri par byes est le dernier appliqué, donc le plus fort : un joueur qui a déjà été exempté passe en tête de liste et sera apparié en priorité. C'est la règle « pas de second bye tant que d'autres n'en ont pas eu », appliquée à l'intérieur d'un groupe (l'équité entre groupes différents n'est pas assurée : point ouvert) ;
- pairing: "wins" ne fait qu'ordonner : il ne calcule aucun score de type Buchholz. Il rapproche les joueurs de même palmarès dans un même groupe de défaites ;
- avoid_clubs est une préférence, pas une contrainte : si aucun adversaire d'un autre club n'est disponible, la seconde passe accepte le même club ;
- l'interdiction de rematch est en revanche une **contrainte dure**, sauf allow_rematch ou appel de repli (allowRematch = vrai).

10.5 Proposition en mode continu

```
func (s *State) proposeSwiss(ph *PhaseState) []Action
```

```
si swissDone(ph) → nil
```

```
si Mode == "rounds" → proposeSwissRound(ph)
```

```
rng := s.rng() ; L := Cfg.Lives ; free := free(ph)
```

```
budget := +∞
```

```
si Target > 0 :
```

```
    budget := sumLives - runningInPhase - Target ← matchs encore lançables
```

```

    si budget <= 0 → nil                                     ← la phase est figée

pour l de 0 à L-1, tant que budget > 0 :
    g := les joueurs libres ayant exactement l défaites
    pairs, _ := pairGroup(ph, g, rng, faux)
    pour chaque paire, tant que budget > 0 :
        action start_match
            Label = "<l> défaite(s), match <Wins[a]+Losses[a]+1>"
            A, B = la paire ; Length = swissLength(ph)
        budget--

si aucune action, aucun match en cours, et au moins 2 joueurs libres → REPLI

swissLength(ph) vaut Cfg.LengthLate quand la phase porte un late_threshold et qu'il ne reste pas plus
de joueurs en vie que lui — la fin d'un suisse se joue en matchs plus longs. Un length_changed posé à la
main par le TD (ph.Length != Cfg.Length) l'emporte : c'est lui qui dirige.

Le budget garantit qu'on ne « dépasse » jamais la cible : avec  $\Sigma$  vies = 18 et target 16, un seul match peut
être lancé, même si quatre joueurs sont libres.

Repli (quand aucun appariement régulier n'est possible et qu'aucun match ne tourne) :

1. pour l de 0 à L-1 :
    pairs := pairGroup(ph, groupe l, rng, allowRematch = VRAI)
    si une paire existe → une seule action, Label = "<...> (rematch)"
2. sinon :
    fr := sortedIDs(free) trié stablement par nombre de défaites croissant
    Label = "Finale" si len(fr) == 2, sinon "Match croisé"
    une action opposant fr[0] et fr[1]

Ce repli est ce qui permet de toujours conclure : quand il ne reste que deux joueurs qui se sont déjà
rencontrés, ou deux joueurs de groupes différents (par exemple un à 0 défaite et un à 1 défaite en fin de
suisse à 2 vies), le moteur propose quand même le match. Les libellés Finale et Match croisé sont d'ailleurs
reconnus par les tests d'invariants comme les seules exceptions autorisées à la règle « pas de match entre
groupes différents ».



## 10.6 Proposition en mode par rondes

func (s *State) proposeSwissRound(ph *PhaseState) []Action

Une ronde reste ouverte tant que l'un des joueurs appelés n'y est pas engagé. Une ronde de 12 matchs dans
une salle de 6 tables se lance en deux vagues ; un bye se confirme parfois avant les matchs. Tant que la ronde
est ouverte, ses appariements non lancés restent proposés — les mêmes —, et la ronde suivante ne vient que
lorsque la ronde est entièrement engagée et ses matchs terminés.

R := ph.Round
si R > 0 :
    rest := roundRest(ph, R)                                ← ce qui reste de la ronde ouverte
    si rest non vide → rest
si runningInPhase > 0 → nil                                  ← on attend la fin de la ronde
r := R + 1 ; free := free(ph)
acts, normal := pairRound(ph, r, free, roundRng(ph, r))
si non normal et au moins 2 joueurs libres → repli (identique au continu, générateur rng())
→ acts

pairRound(v, r, joueurs, rng) est l'appariement d'une ronde sur l'état v :

budget := ∞ ; si Target > 0 : budget := sumLives(v) - Target ; si budget 0 → (nil, normal)
pour l de 0 à Lives-1, tant que budget > 0 :
    g := joueurs à l défaites (dans v)
    pairs, rest := pairGroup(v, g, rng, faux)
    pour chaque paire, tant que budget > 0 → start_match, Label = Ronde r, Round = r ; budget--
    restes += rest
si aucune action → (nil, non normal)

```

si budget 0 → (actions, normal) ← ronde tronquée par la bascule : pas de bye
pour chaque joueur de restes → bye, Label = Ronde r, Round = r

Les mêmes appariements, parce qu'ils sont *recalculés à l'identique* plutôt que stockés. Le plan d'une ronde ne dépend que de trois choses :

- l'état **d'avant la ronde** : `beforeRound(ph, r)` est la phase dont on a retiré les victoires, défaites et adversaires des matchs de ronde r (non annulés), et les byes de ronde r (`PhaseState.ByeRounds`);
- les **joueurs appelés** : `PhaseState.Roster`, relevé par `Apply` au premier événement de la ronde (`match_started` ou `bye` dont `Round` dépasse `ph.Round`), avant de l'appliquer — exactement les joueurs libres qu'un `Propose` de cet instant appariait;
- un **générateur propre à la ronde** :

```
func (s *State) roundRng(ph *PhaseState, r int) *rand.Rand {
    return rand.New(rand.NewSource(s.Seed*1000003 + int64(ph.Index)*7919 + int64(r)*104729))
}
```

Il ne dépend pas du nombre d'événements : confirmer le premier match d'une ronde redistribuait sinon les suivants.

`roundRest(ph, R)` recalcule le plan `pairRound(beforeRound(ph, R), R, ph.Roster, roundRng(ph, R))` et garde :

- chaque `start_match` dont aucun des deux joueurs n'est **engagé** dans la ronde (un match non annulé de ronde R, ou un bye de ronde R), ni éliminé ou retiré, ni occupé; sa longueur est celle d'aujourd'hui (`swissLength`), un `length_changed` ayant pu passer;
- chaque `bye` dont le joueur n'est pas engagé.

Les **orphelins** — joueurs du plan prévus contre quelqu'un qui a joué autre chose (un match lancé à la main) ou qui est parti — sont appariés entre eux par `pairGroup`, dans le budget de la bascule s'il y en a une (`sumLives - matchs en cours - Target - matchs déjà gardés`). Un orphelin seul attend la ronde suivante. Un joueur inscrit pendant la ronde n'est pas dans `Roster` : il entre à la ronde suivante.

Limites assumées : un joueur retiré en cours de ronde, dans une phase à bascule, change le budget et peut donc changer les appariements non lancés; la ronde reste une ronde (personne n'y joue deux fois).

Le repli du mode par rondes :

```
func (s *State) proposeSwissContinuousFallback(ph, free, rng) []Action
```

1. pour l de 0 à `Lives-1` : une paire avec `allowRematch = VRAI` → Label = `rematch`
2. sinon : fr trié par défaites croissantes → une action, Label = `final`

Les actions du repli ne portent pas de numéro de ronde (`Round = 0`) : elles ne comptent dans aucune ronde.

Le compteur de rondes `ph.Round` n'est pas incrémenté par le moteur : il suit le champ `Event.Round` des événements `match_started` et `bye` (relu du libellé "Ronde k" pour un journal de version 0). Dès que le TD confirme la première action d'une ronde, `ph.Round` prend son numéro et `Roster` est relevé. `Match.Round` garde la ronde de chaque match.

10.7 Classement d'une phase à vies

```
func (s *State) livesRanking(ph *PhaseState) []Rank
```

Utilisé pour `swiss_lives` et `gsl` (et pour toute phase dont le format n'a pas de classement propre).

```
alive := alive(ph)
```

pour chaque entrant p :

```
    score := Wins[p]
```

```
    note := "<Wins> victoires, <Losses> défaites"
```

```
    si remainingLives(p) > 0 :
```

```
        score += 1000 + remainingLives(p)
```

```
        note := "en vie (<n> vies)"
```

```
        si len(alive) == 1 → note := "vainqueur"
```

```
    si p est retiré :
```

```
        score := Wins[p]
```

```
        note := retiré (Wins, Losses, Lives = vies qui lui restaient)
```

```

si len(alive) == 1 et ElimOrder non vide :
    le DERNIER éliminé reçoit score 999 et la note "finaliste"

tri stable par score décroissant
rang := 1 ; pour chaque i : si score[i] != score[i-1] → rang := i+1

Lecture des scores :

```

Score	Signification
1000 + vies + victoires	joueur encore en vie ; plus il a de vies et de victoires, mieux il est classé
999	finaliste (dernier éliminé quand le vainqueur est connu)
victoires	éliminé, ou retiré ; départagé par le nombre de victoires au moment de l'élimination (ou du retrait), ex æquo conservés

C'est la traduction exacte du principe « pas de départage » : deux joueurs éliminés avec le même nombre de victoires sont **ex æquo**, définitivement.

Un joueur retiré est classé sur son parcours, comme s'il avait été éliminé à l'instant de son retrait : par ses victoires, **ex æquo** avec les éliminés qui en ont autant, derrière tous les joueurs encore en vie. Ses vies restantes ne comptent pas — il ne les jouera pas — mais la note les porte, avec ses victoires et ses défaites (**withdrawn**). Le parti pris est celui des éliminés : pas de départage, et un retrait n'efface pas ce qui a été joué. Le meilleur joueur d'un suisse de cinq jours qui part le mercredi avec huit victoires était classé dernier (« forfait ») ; il est désormais classé parmi les éliminés à huit victoires. Le « forfait » ne désigne plus que le match abandonné (**Match.Forfeit**) et, dans le classement propre d'une section, le match perdu par forfait (**forfeit**).

11 Format gsl — blocs de groupes

11.1 Principe

Le format GSL est un suisse à 2 vies dont **l'appariement est figé par blocs** : au lieu d'apparier au fil de l'eau (manipulable par l'heure d'annonce d'un résultat), on constitue des groupes de 4 (ou 3, ou 2) joueurs et on joue le bloc entier avant d'en constituer un nouveau.

- Les joueurs à **0 défaite** sont répartis en groupes GSL (double élimination interne à 4 : ouverture, gagnants, perdants, décisif).
- Les joueurs à **1 défaite** (une seule vie) sont répartis en mini-tableaux à élimination directe.
- **Lives** est forcé à 2 par la validation.

C'est la parade anti-manipulation recommandée par l'étude, et elle a l'avantage de se caler naturellement sur les créneaux horaires (un bloc par créneau, repas compris).

11.2 Partition d'une liste en groupes

```
func partition(g []PlayerID) [][]PlayerID
```

La liste (**déjà mélangée** par l'appelant) est découpée en groupes de 4 autant que possible, le reste étant absorbé selon $n \bmod 4$:

$n \bmod 4$	Tailles produites
0	4, 4, ..., 4
3	4, ..., 4, 3
2	4, ..., 4, 3, 3 si $n \equiv 2 \pmod 4$; sinon 2
1	4, ..., 4, 3, 2 si $n \equiv 1 \pmod 4$; sinon 1

Les groupes sont ensuite découpés dans l'ordre de la liste (`g[k:k+taille]`).

Un groupe de taille 1 (`n == 1`) n'est pas transformé en section : le joueur passe le bloc sans jouer, sans perdre de vie ni gagner de by.

11.3 Fin de bloc, fin de phase

```
func (s *State) gslBlockDone(ph *PhaseState) bool
```

Tous les GMatch des sections dont `Block == ph.Round` sont Done.

```
func (s *State) gslDone(ph *PhaseState) bool
```

```
si runningInPhase > 0 ou non gslBlockDone → faux
si Target > 0 et sumLives <= Target      → vrai
→ len(alive) <= 1
```

11.4 Proposition

```
func (s *State) proposeGSL(ph *PhaseState) []Action
```

```
si gslDone(ph) → nil
```

si le bloc courant n'est pas terminé :

```
→ pour chaque match de readyFree(ph) dont la section a Block == ph.Round :
    action start_match
        Section = nom de la section, Key = clé du GMatch
        Label   = "Bloc <block>, <section> : <libellé du GMatch>"
        Length  = longueur du GMatch
```

si `runningInPhase > 0` → nil ← finale ou matchs hors bloc en cours

nouveau bloc :

```
rng := s.rng()
g0 := joueurs en vie à 0 défaite
g1 := joueurs en vie à 1 défaite ou plus
```

si `len(g0) == 1` et `len(g1) == 1` :

```
→ action start_match, Label = "Finale", sans section ni clé
```

sinon :

```
groupes := []
pour g dans [g0, g1] :
    g := sortedIDs(g) puis rng.Shuffle(g)
    groupes += partition(g)
→ action draw, Label = "Bloc <ph.Round+1> : <n> groupes",
    Draw = { Groups: groupes, Lives: vies des entrants }
```

La finale GSL est un match libre (sans section) opposant le dernier joueur invaincu au dernier joueur à une défaite. Si l'invaincu gagne, l'autre atteint 2 défaites et la phase se termine. S'il perd, les deux joueurs ont une défaite chacun, `alive` vaut toujours 2, et le moteur repropose une finale : c'est une **recharge implicite** exactement équivalente à celle de la double élimination. C'est la raison pour laquelle un GSL à 64 joueurs produit 126 ou 127 matchs.

11.5 Application du tirage

```
func (s *State) applyGSLDraw(ph *PhaseState, d *Draw) error
```

```
ph.Round++ ; ph.Drawn = true
```

pour chaque groupe `g` d'indice `i` :

```
si len(g) < 2 → passer
nom := "B<ph.Round>G<i+1>"
si Losses[g[0]] == 0 → sec := gslSection(nom, g, ph.Length)
sinon                → sec := seSection(nom, g, ph.Length)
sec.Block, sec.Group = ph.Round, i+1
```

```
ph.Sections += sec
ph.resolve(s.Withdrawn)
```

Le type de section est déterminé par le **premier joueur du groupe** : c'est correct puisque la partition est faite séparément sur `g0` et `g1`, un groupe est donc homogène en nombre de défaites.

Le classement d'une phase GSL est `livesRanking` (chapitre précédent).

12 Formats bracket et lives_bracket — tableaux

Les deux formats partagent tout le code ; ils ne diffèrent que par les vies à l'entrée, donc par le tirage :

- **bracket** : tous les entrants ont 1 vie ; le tableau est un tableau classique complété par des exemptions.
- **lives_bracket** : les entrants arrivent avec les vies héritées de la phase précédente ; un joueur à 2 vies occupe une paire entière et est donc **exempt du premier tour**.

12.1 Utilitaire

```
func pow2ceil(n int) int // la plus petite puissance de 2 >= n (1 pour n <= 1)
```

12.2 Tirage des places

```
func drawSlots(entrants []PlayerID, lives map[PlayerID]int, rng *rand.Rand) []PlayerID
ids := sortedIDs(entrants) puis rng.Shuffle(ids)
```

```
deux := les ids avec lives >= 2 (comptent 2 dans sum)
une := les autres (comptent 1 dans sum)
size := max(2, pow2ceil(sum))
nPairs := size / 2
```

```
ordre := indices pairs croissants (0, 2, 4, ...) suivis des indices impairs (1, 3, 5, ...)
k := 0
pour chaque p de `deux` :
    pairs[ordre[k]] = (p, BYE) ; marquée occupée ; k++
```

```
extra := size - sum // nombre d'exemptions supplémentaires à distribuer
rest := `une`
pour chaque paire i non occupée, dans l'ordre croissant :
    a, b := BYE, BYE
    si rest non vide → a := dépiler(rest)
    si extra > 0 et a != BYE → extra-- // cette paire reçoit une exemption
    sinon si rest non vide → b := dépiler(rest)
    pairs[i] = (a, b)
```

```
slots := aplatissage des paires : (p0 , p0 , p1 , p1 , ...)
```

Trois idées :

1. **La taille du tableau est dictée par la somme des vies**, pas par le nombre de joueurs : c'est ce qui rend la bascule depuis un suisse exacte quand Σ vies est une puissance de 2.
2. **Les exemptés sont écartés** : en remplissant d'abord une paire sur deux (indices pairs), on évite que deux joueurs à 2 vies se retrouvent dans la même moitié de bracket de premier tour.
3. **Les exemptions surnuméraires** (`extra`) vont en seconde position des premières paires libres, donc aux joueurs placés en tête de la liste mélangée — c'est-à-dire au hasard.

12.3 Construction du tableau

```
func (s *State) buildBracket(ph *PhaseState, slots []PlayerID)
```

```

main := bracketSection("main", "main", slots, bracketLengths(ph))
Sections := [main]

si Cfg.Consolation et len(main.Rounds) >= 2 :
    conso := consolationSection("conso", main, ph.Length)
    Sections += conso

    si Cfg.LastChance et len(conso.Rounds) >= 3 :
        srcs := pour r de 0 à len(conso.Rounds)-3 :
            les perdants de tous les matchs de conso.Rounds[r]
        Sections += bracketFromSrcs("last", "last", srcs, ph.Length)

    si Cfg.Reconciliation :
        gf := section "gf"
        gf.0 « Grande finale » : vainqueur du dernier match de main
            contre vainqueur du dernier match de conso
            longueur = bracketLengths(ph).at(0, 1) - la longueur de finale
        si Cfg.Recharge :
            gf.1 « Grande finale (recharge) » : vainqueur de gf.0 contre perdant de gf.0
                Cond = vrai, CondFrom = 0, CondSide = 1
            Rounds = [[0]] (+ [[1]] si recharge)
            Sections += gf

ph.Drawn = true
ph.resolve(s.Withdrawn)

```

La **dernière chance** ne recueille que les perdants des tours de consolante **sauf les deux derniers** : perdre en finale ou en demi-finale de consolante ne donne pas droit à un troisième tableau.

La **recharge** est le mécanisme de la vraie double élimination : `CondSide = 1` signifie que le match n'existe que si le vainqueur de la grande finale est le joueur venu de la place 1, c'est-à-dire le vainqueur de la consolante (qui a déjà une défaite). Le vainqueur du principal, invaincu, doit donc être battu deux fois. Si c'est lui qui gagne la première grande finale, `resolve` marque `gf.1` comme `Skipped` (donc `Done`) et le tournoi se termine.

12.4 Application du tirage

```
func (s *State) applyDraw(ph *PhaseState, section string, d *Draw) error
```

Pour `bracket` et `lives_bracket` :

```

si len(d.Slots) < 2 ou len(d.Slots) n'est pas une puissance de 2
    → erreur « le nombre de places doit être une puissance de 2 »
buildBracket(ph, d.Slots)

```

12.5 Proposition

```
func (s *State) proposeBracket(ph *PhaseState) []Action
```

```

si non ph.Drawn :
    si len(Entrants) < 2 → nil
    slots := drawSlots(Entrants, Lives, s.rng())
    → une action draw
        Label = "<nom de la phase> : tableau de <len(slots)> places"
        Draw = { Slots: slots, Lives: vies des entrants }

sinon :
    pour chaque match de readyFree(ph) :
        Label := libellé du GMatch
        si la section est "main" et qu'il existe plus d'une section
            → Label := "Principal, " + libellé
        → action start_match { Section, Key, Label, A, B, Length du GMatch }

```

Toutes les sections sont proposées en parallèle : consolante, dernière chance et principal avancent simultanément dès que les places sont connues et les joueurs libres.

12.6 Classement d'un tableau

func (s *State) bracketRanking(ph *PhaseState) []Rank

Priorité des sections (plus haut = mieux classé) :

Section	prio
gf (grande finale)	4
main, se	3
conso	2
last	1

si non ph.Drawn :

→ tous les entrants au rang 1, note « en attente du tirage »

initialiser score[p] = -1, note = « non classé » pour chaque entrant

(1) SORTIES - pour chaque section, chaque tour r, chaque match g de ce tour :

ignorer si Skipped

si non Done : chaque joueur RETIRÉ placé dans g y prend sa sortie (il le perdra par forfait)

ignorer si Walkover et que le perdant n'est pas retiré (exemption par BYE)

candidat := prio[section]*1000 + r ← pour le PERDANT

remplacer score[g.Loser] si :

score actuel < 0

OU prio[section] < (score actuel)/1000 ← sortie en section moins prioritaire

OU (prio identique ET candidat > score actuel) ← sortie plus profonde

note := "<nom de section>, <libellé du match>"

(2) VAINQUEURS DE SECTION - pour chaque section non vide :

last := dernier match de la section

(pour "gf" : le dernier match Done et non Skipped, en partant de la fin)

si last.Done, non Skipped, et Winner != BYE :

candidat := prio[section]*1000 + 999

remplacer si score < 0, ou candidat > score, ou section == "gf"

note := "vainqueur <nom de section>"

si section == "gf" : le perdant reçoit prio(gf)*1000+998, note « finaliste »

(3) RETIRÉS ET NON CLASSÉS :

si retiré → note := retiré (withdrawn), avec la section et le libellé de sa sortie s'il en a une
(son score, lui, est celui de son parcours)

si score < 0 :

score := 0

si la note est restée « non classé » → note « en cours », score := 5000

tri stable par score décroissant, rangs avec ex æquo

La règle (1) est contre-intuitive mais essentielle : **un joueur est classé par sa dernière sortie**, c'est-à-dire par la section de plus faible priorité où il a perdu. Un joueur battu au premier tour du principal puis en finale de consolante est classé sur sa performance en consolante, pas sur son élimination du principal.

Un joueur **retiré** est classé comme s'il perdait chacun de ses matchs restants : le match qu'il perd par forfait (walkover résolu contre lui) est une sortie comme une autre, et la règle (1) retient la dernière. Il est donc **ex æquo avec les perdants du match qui l'attendait**. Un joueur de `lives_bracket` qui gagne un match puis part est classé avec les perdants du tour de consolante où il redescend, et non dernier ; une demi-finaliste qui part pendant sa demi-finale, dans un tableau à consolante, est classée avec la perdante du tour de consolante qu'elle aurait joué — et non devant la gagnante de la consolante, qui a joué deux matchs de plus. Tant que l'adversaire de ce match n'est pas connu, le match qui l'attend tient lieu de sortie : le retiré n'est jamais « en cours ». Retiré sans avoir joué, il est classé avec les perdants de son premier match (#25).

Le score 5000 attribué aux joueurs « en cours » les place **devant tout le monde** : c'est voulu, puisque le classement intermédiaire doit montrer les joueurs encore en course en tête.

12.7 Survivants d'un tableau

```
func (s *State) bracketSurvivors(ph *PhaseState) []PlayerID

si non ph.Drawn      → tous les entrants
r := bracketRanking(ph)
si ph.allDone()      → [ r[0].Player ]      ← le vainqueur seul
sinon                → tous les joueurs dont la note est « en cours »
```

13 Format round_robin — poules

13.1 Principe

Les entrants sont répartis en poules ; chaque poule est une toutes rondes (table de Berger). Les **qualifiers** premiers de chaque poule passent à la phase suivante. Les égalités à la place qualificative ne sont **pas** départagées arithmétiquement : elles déclenchent un **barrage** à 2 vies entre les joueurs concernés.

13.2 Tirage des poules

```
si non ph.Drawn :
    n := len(Entrants) ; si n < 2 → nil
    ids := sortedIDs(Entrants) puis rng.Shuffle(ids)
    ng := ceil(n / GroupSize) (au minimum 1)
    répartir en « serpentín simple » : groups[i mod ng] += ids[i]
    → une action draw, Label = "<ng> poules", Draw = { Groups }
```

La distribution $i \bmod ng$ équilibre les tailles à un joueur près.

```
func (s *State) applyRRDraw(ph *PhaseState, section string, d *Draw) error
```

Avec section vide (tirage des poules) :

```
pour chaque groupe i :
    sec := rrSection("Poule <lettre A+i>", g, ph.Length) ; sec.Group = i+1
ph.Drawn = true ; ph.resolve(...)
```

Les poules sont nommées Poule A, Poule B, ... (`groupName(i) = "Poule " + ('A'+i)`). Au-delà de 26 poules, le caractère produit sort de l'alphabet : limite connue, sans conséquence fonctionnelle.

13.3 Décompte et détection des égalités

```
func rrWins(sec *Section) map[PlayerID]int
```

Compte les victoires **dans la section** : chaque joueur figurant en `Src[k].Player` est initialisé à 0 (pour qu'un joueur sans victoire apparaisse), puis chaque match **Done** et **non Walkover** ajoute une victoire à son vainqueur. Les walkovers (exemptions, forfaits) ne comptent donc pas.

```
func (s *State) rrTie(ph *PhaseState, sec *Section) (tied []PlayerID, spots int)

w := rrWins(sec)
ids := sortedIDs(clés de w) puis tri stable décroissant sur w
q := Cfg.Qualifiers
si q >= len(ids)      → pas d'égalité (toute la poule est qualifiée)
si w[ids[q-1]] != w[ids[q]] → pas d'égalité (la coupure est nette)

v := w[ids[q-1]]      ← le nombre de victoires litigieux
spots := q
pour chaque p :
    si w[p] > v → spots--      ← qualifié d'office, il consomme une place
    si w[p] == v → tied += p
```

Résultat : **tied** sont les joueurs à égalité sur la ligne de coupure, **spots** le nombre de places qu'ils doivent se partager.

13.4 Barrage

Un barrage est une Section d'un genre particulier : elle n'a **pas de graphe** (Matches vide) ; ses matchs sont appariés dynamiquement, comme un suisse à 2 vies, jusqu'à ce qu'il ne reste que **Spots** joueurs.

```
func (s *State) applyRRDraw(...) // avec section = "Barrage <nom de poule>"

si len(d.Groups) != 1 → erreur « un seul groupe attendu »
poule := ph.section(nom sans le préfixe "Barrage ")
si absente → erreur
_, spots := rrTie(ph, poule) ← les places sont RECALCULÉES, pas relues du tirage
ph.Sections += Section{Name, Kind: "barrage", Players: d.Groups[0], Spots: spots}
```

```
func (s *State) barrageLosses(bs *Section) (losses map[PlayerID]int, running int)
```

Parcourt MatchOrder ; pour les matchs de section bs.Name non annulés : compte les Running et incrémente losses[perdant] pour les autres.

```
func (s *State) barrageAlive(bs *Section) []PlayerID // moins de 2 défaites
```

```
func (s *State) proposeBarrage(ph *PhaseState, bs *Section) []Action
```

```
losses, running := barrageLosses(bs)
alive := barrageAlive(bs)
budget := len(alive) - running - bs.Spots
si budget <= 0 → nil ← le barrage a rendu son verdict
```

```
free := alive privés des joueurs occupés
pour l de 0 à 1, tant que budget > 0 :
    g := free ayant exactement l défaites de barrage
    g := sortedIDs(g) puis rng.Shuffle(g)
    apparier g[0]-g[1], g[2]-g[3], ... tant que budget > 0
        action start_match { Section: bs.Name, Label: bs.Name }
```

```
si aucune action, running == 0 et au moins 2 joueurs libres :
    fr := sortedIDs(free) trié par défaites croissantes
    → une action croisée fr[0] contre fr[1], Label = "<nom> (croisé)"
```

Même logique de budget que le suisse : on ne lance pas un match qui ferait descendre le nombre de survivants en dessous du nombre de places.

```
func (s *State) rrBarragesDone(ph *PhaseState) bool
```

Pour chaque poule ayant une égalité : le barrage doit exister et len(barrageAlive) <= Spots.

13.5 Proposition

```
func (s *State) proposeRR(ph *PhaseState) []Action
```

si non ph.Drawn → l'action de tirage des poules (ci-dessus)

```
acts := pour chaque match de readyFree(ph) :
    action start_match { Section, Key, Label = "<section>, <libellé>" }
```

```
pour chaque section de genre "poule" dont TOUS les matchs sont faits :
    tied, spots := rrTie(ph, sec)
    si pas d'égalité → passer
    si la section « Barrage <poule> » n'existe pas encore :
        acts += action draw
            Section = "Barrage <poule>"
            Label   = "Barrage <poule> : <n> joueurs pour <spots> place(s)"
            Draw     = { Groups: [tied] }
```

```

sinon :
    acts += proposeBarrage(ph, barrage)

```

Les poules qui ont fini avancent donc vers leur barrage pendant que les autres jouent encore.

13.6 Qualifiés et classement

```

func (s *State) rrQualified(ph *PhaseState) []PlayerID

pour chaque poule :
    w := rrWins ; ids := sortedIDs puis tri décroissant sur w
    tied, spots := rrTie
    si pas d'égalité :
        → les `Qualifiers` premiers de ids
    sinon :
        v := w[tied[0]]
        → tous ceux qui ont strictement plus de v victoires
        + si un barrage existe et len(barrageAlive) <= spots → les survivants du barrage

func (s *State) rrRanking(ph *PhaseState) []Rank

qual := ensemble des rrQualified
pour chaque joueur d'une poule :
    score := victoires dans sa poule
    note := "<Poule X>, <n> victoires"
    si retiré → note := retiré (withdrawn), avec la poule et ses victoires ; jamais qualifié
    sinon, si qualifié → score += 100 ; note += ", qualifié"

ids := sortedIDs(Entrants) trié stablement par score décroissant
rangs avec ex æquo (même score → même rang)

```

Le bonus de 100 place tous les qualifiés devant tous les non-qualifiés, quelles que soient les poules — ce qui est correct puisque les poules ne sont pas comparables entre elles. Un retiré n'en bénéficie pas : il est classé sur ses victoires, ex æquo avec les non-qualifiés qui en ont autant (#25).

14 Classement général et prix

14.1 Concaténation des phases

```

func (s *State) Ranking() []Rank

```

Le classement général se lit **de la dernière phase atteinte à la première** : les joueurs éliminés plus tôt sont classés derrière.

```

out := [] ; vus := {} ; offset := 0
pour i de la dernière phase à la première :
    r := phaseRanking(Phases[i])
    pour chaque rang rk de r :
        si rk.Player déjà vu → passer      ← déjà classé par une phase plus tardive
        marquer vu
        out += { Player, Rank: rk.Rank + offset, Note: rk.Note }
    offset += len(r)                        ← len(r), pas le nombre de nouveaux !

tri stable par Rank croissant
renumérotation dense en conservant les ex æquo :
    pos := 0 ; prev := -1 ; prevRank := 0
    pour chaque élément :
        si Rank != prev → prev := Rank ; prevRank := pos+1
        Rank := prevRank ; pos++

```

L'offset s'incrémente de la **taille complète** du classement de phase (y compris les joueurs déjà vus) : cela garantit qu'aucun joueur d'une phase antérieure ne peut se glisser devant un joueur d'une phase postérieure,

quelle que soit la répartition des ex æquo.

La renumérotation finale produit un classement dense : 1, 2, 2, 4, 5, 5, 5, 8... (rang = position du premier élément du groupe d'ex æquo).

```
func (s *State) phaseRanking(ph *PhaseState) []Rank
```

Kind	Fonction
bracket, lives_bracket	bracketRanking
round_robin	rrRanking
swiss_lives, gsl	livesRanking

14.2 Prix

```
func Prizes(ranking []Rank, prizes []float64) map[PlayerID]float64
```

Les prix sont indexés par place : `prizes[0]` est la dotation du premier, `prizes[1]` du deuxième, etc. Les ex æquo **partagent à parts égales les prix des places qu'ils occupent** :

grouper le classement par Rank

pour chaque groupe de rang `r` comptant `k` joueurs :

total := somme de `prizes[r-1] ... prizes[r-2+k]` (les indices hors tableau valent 0)

chaque joueur du groupe reçoit `total / k`

Exemple : deux joueurs ex æquo au rang 3, dotations [100, 60, 40, 20, 10] → ils se partagent 40 + 20 = 60, soit 30 chacun ; le joueur suivant est au rang 5 et reçoit 10.

L'unité est libre (montant, pourcentage, part) : le moteur ne fait qu'additionner et diviser. Cette fonction est le partage entre ex æquo, et rien d'autre ; d'où viennent les montants par place est décrit ci-dessous.

14.2.1 Pool, retenue, barème par section

```
type PrizeScale struct {
    Percents []float64 `json:"percents,omitempty"` // % du pool distribuable, place par place
    Amounts   []float64 `json:"amounts,omitempty"`  // montants fixes, place par place
}
```

```
type Retention struct {
    Amount float64 `json:"amount,omitempty"`
    Percent float64 `json:"percent,omitempty"`
}
```

```
type PrizePool struct {
    EntryFee float64 `json:"entry_fee,omitempty"`
    Retention Retention `json:"retention,omitempty"`
    Sections map[string]PrizeScale `json:"sections,omitempty"`
}
```

```
const PrizeSectionAll = "all" // clé du classement GÉNÉRAL dans Sections
```

```
func (s *State) Pool() float64
func (s *State) Distributable() float64
func (s *State) PrizeAmounts(section string) []float64
func (s *State) SectionPrizes(section string) map[PlayerID]float64
func (s *State) SectionRanking(section string) []Rank
```

Une affiche de tournoi annonce « 50/30/20 % » et non « 216/130/86 € » : le pool dépend du nombre d'inscrits, qu'on ne connaît qu'à la clôture des inscriptions.

```
Pool() = EntryFee × nombre d'inscrits (les retirés comptent : ils ont payé)
Distributable() = Pool - Pool × Retention.Percent/100 - Retention.Amount (jamais négatif)
```

`PrizeAmounts(sec)` : si le barème donne des `Amounts`, ce sont eux, tels quels. S'il donne des `Percents`, chaque place vaut `Distributable × pct/100`, **arrondie à l'unité** — personne ne paie en centimes à une table de tournoi — et le `RESTE`, en plus ou en moins, va au **premier**. C'est la seule façon de garantir que la somme distribuée égale exactement le pool après retenue, et le premier prix est celui où un euro d'écart se voit le moins.

`Sections` est indexée par identifiant de section (`main`, `conso`, `last`, `gf`, `poule:A...`), plus la clé réservée `all` pour le classement général — celui que `Ranking()` renvoie, et le seul qui existe dans un tournoi sans tableau.

`SectionRanking(sec)` est le classement PROPRE d'une section : ses joueurs, par tour atteint dans cette section, le vainqueur en tête. Il diffère du classement général, qui mélange les sections par priorité (un vainqueur de consolante passe derrière un demi-finaliste du principal). Une section de poule ou de barrage se classe par nombre de victoires, faute de tour atteint. Un retiré dont le prochain match attend encore son adversaire est classé au tour de ce match (note `withdrawn`), et non « en cours ». La section est cherchée de la dernière phase vers la première.

`Validate` refuse une dotation incohérente : pourcentages ET montants dans le même barème, pourcentages totalisant plus de 100 % du pool, retenue hors [0, 100] %, montant négatif. Une dotation fausse ne se voit qu'au moment de payer, devant les joueurs.

La forme ancienne (`"prizes": [100, 60, 40]`, une simple liste de montants) reste lue et devient le barème du classement général : les journaux existants n'ont pas à être réécrits.

14.3 Export CSV

```
func (s *State) StandingsCSV() []byte
```

Séparateur `;`, en-tête `section;rang;id;nom;club;note;prix`. La première colonne dit à quel classement la ligne appartient : `all` pour le classement général (puis `Final` s'il existe, sinon `Ranking()`), puis l'identifiant de chaque **section dotée**, dans l'ordre alphabétique. Les blocs sont contigus : un tableur les lit comme des tableaux séparés, un hôte qui n'en veut qu'un filtre sur la colonne. Les sections apparaissent parce qu'elles ont une dotation — sortir le classement de chaque groupe GSL d'un tournoi de cent joueurs noierait la feuille affichée au mur. Le prix est formaté avec deux décimales. Un joueur absent de `Players` est écrit avec son identifiant en guise de nom et un club vide.

15 Horloge et prévisions

```
func (s *State) Expected(n int) time.Duration
```

`Config.MinPerPoint × n` minutes. Avec le défaut de 8 min/point, un match en 7 points est attendu en 56 minutes. L'étude recommande de compter **11 min/point pour planifier une ronde** (attente comprise) et 8 min/point pour l'espérance d'un match.

```
func (s *State) SlowMatches(now time.Time, facteur float64) []*Match
```

Les matchs en cours dont la durée écoulée dépasse `facteur × Expected(Length)`. Le rendu HTML utilise un facteur de 1,5 pour colorer une ligne en rouge.

```
type Clock struct {
    Played   int           `json:"played"`
    Running  int           `json:"running"`
    Elapsed  time.Duration `json:"elapsed"`
    AvgMatch time.Duration `json:"avg_match"`
    AvgPerPt time.Duration `json:"avg_per_pt"`
}
```

```
func (s *State) ClockAt(now, start time.Time) Clock
```

```
Elapsed := now - start
```

```
pour chaque match :
```

```
    Finished, non Forfeit, End renseignée → Played++ ; total += End-Start ; pts += Length
```

```
    Running → Running++
```

```
AvgMatch := total / Played      (si Played > 0)
```

AvgPerPt := total / pts (si pts > 0)

Les matchs gagnés par forfait sont exclus des moyennes : ils fausseraient la mesure.

16 Paquets annexes

16.1 sim — simulation et prévision

Le paquet `sim` joue des tournois fictifs avec le moteur. Il sert à trois choses : les tests d’invariants, la validation des formats (comparaison avec le simulateur de l’étude) et la prévision de l’heure de fin d’un tournoi en cours.

16.1.1 Modèle probabiliste

```
func PGain(a, b float64, n int) float64
```

Probabilité que le joueur de PR `a` batte celui de PR `b` en `n` points, formule Elo/FIBS avec la conversion **3 PR = 100 points Elo** :

$$D = \frac{(b - a) \times 100}{3} \quad P = \frac{1}{1 + 10^{-D\sqrt{n}/2000}}$$

Justification (voir docs/etude_formats.md) : sur la base BMAB (33 238 matchs uniques), 1 PR d’écart vaut 2,8 % à 7–11 points (= 0,036 pour = 0,5 ; théorie 0,039). L’exposant de la longueur n’est pas identifiable avec ces données — la vraisemblance est plate entre 0 et 0,7, = 1 est exclu — mais la mécanique « erreur cumulée N, chance $\sqrt{N}$ » soutient = 0,5.

```
func Duree(n int, minPerPoint float64, rng *rand.Rand) time.Duration
```

Durée d’un match de `n` points : loi **gamma de forme 8** et de moyenne `minPerPoint × n`, obtenue en moyennant huit tirages exponentiels. La forme 8 donne un coefficient de variation d’environ 35 %, proche de l’observation.

16.1.2 Exécution d’un tournoi

```
type Options struct {
    Seed          int64
    MinPerPoint   float64           // défaut 8
    Start         time.Time         // défaut 2026-01-01 09:00 UTC
    MaxSteps      int              // défaut 100000
    Hook          func(*tournoi.State, tournoi.Event) error
}
```

```
type Result struct {
    State      *tournoi.State
    Journal    tournoi.Journal
    Winner     tournoi.PlayerID
    Minutes    float64
    NMatches   int
    Steps      int
    Err        error
}
```

```
func Run(cfg tournoi.Config, players []tournoi.Player, opt Options) Result
```

Boucle de simulation :

créer le tournoi, inscrire tous les joueurs

tant que non terminé :

 acts := Propose()

 pour chaque action non-Wait :

 événement := EventFromAction ; Apply ; ajouter au journal ; appeler Hook

```

    si start_match → enregistrer sa fin prévue (now + Duree(...))
    si draw, next_phase ou finish → SORTIR de la boucle (l'état a changé)
    si des actions ont été appliquées et la première n'était pas Wait → recommencer
    si aucun match en cours et rien n'a progressé → erreur « blocage »
    sinon : avancer l'horloge jusqu'à la fin du match le plus proche,
            tirer son vainqueur selon PGain, appliquer le résultat

```

Le score du vainqueur est `n` ; celui du perdant est tiré uniformément dans `[0, n-1]`.

Hook est appelé après chaque événement appliqué : c'est le point d'accroche des tests d'invariants. Une erreur renvoyée par le hook interrompt la simulation.

```
func Champ(P int, mu, sigma, min, max float64, rng *rand.Rand) []tournoi.Player
```

Crée `P` joueurs de `PR` tirés dans une loi normale $N(\mu, \sigma)$ tronquée à `[min, max]`, **triés du meilleur au moins bon** : `P1` est toujours le meilleur joueur du champ. Les clubs sont affectés cycliquement (`Club 0` à `Club 4`) pour exercer `avoid_clubs`. Le champ de référence de l'étude est `Champ(64, 6, 2, 2, 10)`.

16.1.3 Prév́ision de fin

```
func Forecast(j tournoi.Journal, now time.Time, K int, minPerPoint float64,
    seed int64) ([]float64, error)
```

Rejoue le journal, puis simule `K` fins de tournoi à partir de `now`. Renvoie les durées restantes en minutes, **triées croissantes** : la médiane est `out[K/2]`, le quantile 90 % `out[9K/10]`.

Particularités :

- les `PR` inconnus (0) sont remplacés par la moyenne des `PR` connus, ou 6 s'il n'y en a aucun ;
- pour un match déjà en cours, la durée restante est un tirage neuf moins le temps déjà écoulé, **plancher à 10 % de la durée attendue** ;
- un tournoi déjà terminé renvoie `[0]`.

16.2 render — affichage

Fonctions **pures** produisant du HTML et du SVG. Rien n'y est écrit en dur dans une langue : le moteur ne produit que des codes, et ce paquet les traduit par un **Labeler** que l'hôte fournit — y compris ses propres mots (« Joueur », « Table », « libre »), qui sont eux aussi des codes (**Term**). `French()` est fourni pour la console, la démo et les tests.

```

type Renderer struct {
    L      Labeler
    Style  string // vide = DefaultStyle
    Credit string // vide = DefaultCredit
    Lang   string // attribut lang de la page ; vide = "fr"
}

func New(l Labeler) *Renderer

type Labeler interface {
    Label(tournoi.Label) string
    Note(tournoi.Note) string
    Reason(tournoi.ReasonCode) string
    Warn(tournoi.WarningCode) string
    SectionName(string) string
    PhaseName(tournoi.PhaseConfig) string
    Term(t Term, count int) string
}

func French() Labeler

```

Méthode	Produit
<code>BracketBoardSVG(st, ph)</code>	Toutes les sections en graphe d'une phase dans un seul SVG, côte à côte, avec les descentes d'une section à l'autre

Méthode	Produit
<code>LivesBoard(st, ph, now)</code>	Colonnes de joueurs par nombre de défaites : état, temps d'attente, adversaires déjà rencontrés
<code>TableGrid(st, now)</code>	Une case par table : son match et sa durée, ou libre, indisponible, réservée
<code>RunningTable(st, now)</code>	Les matchs en cours : table, libellé, joueurs, points, durée (classe <code>lent</code> au-delà de $1,5 \times$ attendu)
<code>ActionsList(st, acts)</code>	La liste ordonnée des actions proposées
<code>StandingsTable(st)</code>	Le classement avec club, note et prix
<code>PairingSheet(st, round)</code>	La feuille d'appariements d'un lot, imprimable A4, une case de score par joueur — page autonome
<code>Page(st, acts, now)</code>	La page d'écran de salle, autonome, rafraîchie toutes les 30 s par une méta (pas de script)

Une page produite est autonome : un seul fichier, feuille de style embarquée, aucune ressource externe — pas de police distante, pas de script, pas d'image liée. Elle doit s'ouvrir hors ligne, depuis une clé USB, sur l'ordinateur de la salle. Un crédit figure en pied de page.

Les lots. `Batches(st)` groupe les matchs de la phase courante par instant de départ. Un lot est ce qu'un directeur appelle une ronde dans un suisse par rondes, un bloc dans un GSL, un tour dans un tableau, une micro-ronde dans un suisse à `batch_minutes`. Le moteur n'a pas de mot pour cela et n'en a pas besoin : l'instant de départ le dit. `PairingSheet(st, n)` imprime le n-ième ; 0 ou un numéro au-delà du dernier donne le plus récent.

Mise en page du SVG. Une colonne par tour (`Rounds`), boîte de 190×44 px, écart horizontal 60 px, vertical 12 px, et 40 px entre deux sections. Chaque section occupe sa propre bande de colonnes, si bien que deux arbres ne peuvent pas se chevaucher. L'ordonnée d'un match est la **moyenne des ordonnées de ses matchs sources de la même section** ; les chevauchements sont ensuite corrigés en décalant vers le bas (nécessaire pour les groupes GSL, où le match des gagnants et celui des perdants partagent des sources). Les liaisons sont tracées **sous** les boîtes ; celles qui traversent une section — les descentes du principal vers la consolante — sont en pointillé et d'une autre couleur. Une paire d'exemptions (BYE contre BYE) n'est pas dessinée. Le fond d'une boîte est blanc (match à venir), jaune pâle (en cours) ou vert pâle (terminé).

Le nom affiché d'un joueur est `Player.Name`, ou l'identifiant à défaut — un nom est saisi par le TD et ne se traduit pas. BYE et une place inconnue passent, eux, par le `Labeler`. Tout texte issu des données passe par `html.EscapeString`.

Les rendus sont figés par des **fichiers témoins** dans `render/testdata/` : toute modification du rendu fait échouer les tests, et `go test ./render -update` les régénère une fois le changement relu.

16.3 players — import CSV

```
func FromCSV(b []byte) ([]tournoi.Player, error)
```

Import souple destiné aux exports HelloAsso, FFBG ou tableur :

- BOM UTF-8 retiré ;
- séparateur **déecté** sur la première ligne parmi ; , et tabulation (celui qui apparaît le plus souvent ; , par défaut) ;
- lecture tolérante (`FieldsPerRecord = -1, LazyQuotes`) ;
- colonnes reconnues par **inclusion de sous-chaîne**, en minuscules : `id` ; nom (nom complet, `name`, `nom`) ; prénom (`prénom`, `prenom`, `first`) ; nom de famille (`nom de famille`, `last`) ; club ; PR (`pr`, `rating`, `elo`) ;
- si ni colonne de nom ni colonne de prénom : erreur ;
- le nom est la colonne « nom », sauf si prénom et nom de famille existent tous deux et que le prénom est renseigné — auquel cas c'est leur concaténation ;
- une ligne sans nom est ignorée ;
- l'identifiant est la colonne `id` si elle est renseignée, sinon un **slug** du nom ; en cas de collision, un suffixe -2, -3... est ajouté ;
- le PR accepte la virgule décimale ; une valeur illisible laisse `Rating` à 0.

Le slug conserve les minuscules ASCII, les chiffres et les caractères non-ASCII (donc les lettres accentuées), remplace espace, tiret, souligné et apostrophe par un tiret, supprime le reste, puis élague les tirets aux extrémités.

16.4 Binares de démonstration

16.4.1 cmd/tournoi-demo

Simule un tournoi complet et écrit ses sorties dans un dossier.

```
go run ./cmd/tournoi-demo -format suisse_tableau -joueurs 32 -etapes -sortie demo
go run ./cmd/tournoi-demo -rejouer demo/journal.json
```

Option	Rôle
-format	une clé du catalogue (suisse, suisse_tableau, gsl, elim, conso, double, poules)
-joueurs	effectif (défaut 32)
-graine	graine (défaut 1)
-sortie	dossier de sortie (défaut demo)
-etapes	écrire aussi les pages d’affichage à 15 %, 40 %, 70 % et 100 % du journal
-rejouer	rejouer un journal et afficher l’état, les actions proposées et les avertissements

Fichiers produits : journal.json, affichage.html, classement.csv, un SVG par section, affichage_mi_tournoi.html (rejeu des deux tiers du journal) et, avec -etapes, les pages etapeK*. La console affiche en outre une prévision de fin (Forecast sur 50 tirages) comparée à la durée réellement observée.

16.4.2 cmd/tournoi-td

Console interactive de direction de tournoi, pour éprouver le moteur à la main. Le journal est réécrit après chaque commande et affichage.html régénéré à côté ; relancer la commande sur le même fichier reprend le tournoi.

```
go run ./cmd/tournoi-td -journal montournoi.json -format suisse_tableau
```

Commande	Effet
ajoute <nom> [club] [pr]	inscrire un joueur (identifiant = nom en minuscules)
import <fichier.csv>	inscrire depuis un CSV
propose	afficher les actions proposées, numérotées
ok [n \ tous]	confirmer une proposition, ou toutes
resultat <match> <vainqueur> [scoreA scoreB]	saisir un résultat
corrige <match> <vainqueur> [scoreA scoreB]	corriger un résultat déjà saisi
annule <match>	annuler un match lancé par erreur
forfait <joueur>	retrait du tournoi
longueur <points>	changer la longueur des prochains matchs de la phase
matches, vies, classement, etat	consultations
simule	jouer au hasard tous les matchs en cours
aide, quitte	—

Le joueur peut être désigné par son identifiant ou par une sous-chaîne de son nom (erreur si ambiguë). ok tous s’interrompt après une action draw, next_phase ou finish, puisque l’état a changé et que les propositions suivantes sont caduques.

17 Invariants et plan de tests

17.1 Nature des tests

Les tests **ne sont pas des tests unitaires** : ce sont des **tests d'invariants par simulation**. Un nouveau format ou une nouvelle option s'ajoute dans la liste `configs()` et se trouve automatiquement couvert par toutes les vérifications.

```
go test -short ./...           # la boucle de travail, 2 s : matrice réduite
go test ./...                 # tout : matrice complète et TestParity, 1 min
go test -run TestFormatsInvariants ./
go test -run '^$' -fuzz FuzzApply -fuzztime 60s ./ # fuzzing (aussi en intégration continue)
go vet ./... && gofmt -l .
```

17.2 Invariants vérifiés

`TestFormatsInvariants` joue **toutes les configurations de `configs()` × 8 effectifs (2, 3, 5, 8, 13, 32, 64, 100) × 8 graines**, et vérifie après chaque événement puis à la fin. En mode `-short` la matrice est réduite à six effectifs (2, 3, 5, 8, 13, 32) et trois graines, ce qui la ramène sous les trois secondes : c'est la boucle qu'on lance à chaque sauvegarde, la matrice entière tournant en intégration continue.

1. **Terminaison** : aucune simulation ne dépasse `MaxSteps`, aucun blocage.
2. **Vainqueur unique** : `Final` existe et contient exactement un joueur au rang 1.
3. **Classement complet** : `len(Final) == P` — tous les inscrits sont classés.
4. **Rejeu identique** : `Replay(journal)` produit exactement le même `Final` (comparaison JSON).
5. **Aucun avertissement** après rejeu : `len(Warnings) == 0`.
6. **Aucun joueur éliminé ne joue** : dans une phase à vies, à l'instant où un match démarre, `Losses[p] < Lives[p]` pour les deux joueurs.
7. **Pas de match entre groupes différents** : dans une phase à vies, `Losses[A] == Losses[B]`, sauf pour les libellés `Finale` et `Match croisé` (les replis).
8. **Peu de rematches** : en suisse, le nombre de secondes rencontres reste sous `graines × (Lives + 3)` dès que `P > 8`.

17.3 Fuzzing

Deux cibles gardent `Apply` contre ce qu'aucun test écrit à la main ne cherche :

- `FuzzApply` applique des suites d'événements tirées au hasard. Les octets ne sont pas du JSON mais un **plan** : chaque triplet choisit un type d'événement et ses paramètres parmi ce que l'état contient. Du JSON aléatoire n'atteindrait presque jamais un événement valide, et ne testerait donc que le décodeur.
- `FuzzReplayJournal` rejoue des octets quelconques présentés comme un journal — le fichier que l'hôte a stocké et qui revient tronqué, réordonné ou corrompu.

Dans les deux cas, **une erreur est une réponse acceptable** ; un `panic` ou un état incohérent n'en est pas une. L'état est vérifié après chaque application : autant de matchs que d'identifiants ordonnés et aucun doublon, aucun match d'un joueur contre lui-même, un vainqueur qui appartient à son match, aucun entrant inconnu, aucun compteur négatif, aucun joueur deux fois au classement, et **Propose** qui ne panique pas.

17.4 Compatibilité ascendante

Version	Ce qui change à la lecture
0	libellés en texte français, convertis en codes par <code>Event.upgraded</code> ; ronde relue du libellé
1	codes structurés ; <code>config_changed</code> peut changer les options de construction d'un tableau tiré (sans effet)
2	<code>config_changed</code> ne le peut plus : refus <code>drawn</code>

`testdata/journal_v0.json` est un journal complet écrit dans la forme d'avant les codes structurés — aucun champ `version`, aucun champ `round`, un `label` qui était une chaîne française. Le test le rejoue et compare le classement **place par place**. Cette fixture ne se régénère pas : elle vaut par le fait qu'elle n'a pas bougé.

`TestMatchCounts` fixe les comptes de matchs sur 64 joueurs :

Format	Matchs attendus
Suisse 2 vies continu	126 ou 127
Élimination simple	63
Double élimination avec recharge	126 ou 127
Blocs GSL	126 ou 127

(126 = $2 \times 64 - 2$; le 127 est la recharge, jouée une fois sur deux environ.)

`TestCorrection` vérifie qu’après l’inversion du premier résultat d’un tournoi par `result_corrected`, la somme des défaites de la phase égale le nombre de matchs terminés — c’est-à-dire que `recompute` a bien tout reconstruit.

`TestWithdrawnAfterDraw` vérifie qu’un joueur retiré **après** le tirage d’un tableau n’est plus jamais proposé, que ses matchs non lancés sont perdus par walkover, que le tableau se termine et qu’aucun avertissement n’est produit.

`TestParity` (long, sauté en mode court) mesure sur 1 500 tournois de 64 joueurs par format : P(le meilleur gagne), P(un des quatre meilleurs gagne), le nombre moyen de matchs et la durée moyenne. Ces valeurs sont à comparer à celles de `docs/etude_formats.md`; **tout changement d’algorithme d’appariement doit être validé par ce test**. Ordre de grandeur attendu : une fin de semaine ne fait gagner le meilleur joueur que 4 à 9 % du temps (64 ou 32 joueurs), contre 1,6 à 3 % au hasard.

17.5 Invariants structurels à préserver

Une réimplémentation doit maintenir, à tout instant :

- un joueur a **au plus un match en cours** ;
- `MatchOrder` est en bijection avec les clés de `Matches`, dans l’ordre de lancement ;
- les identifiants de matchs sont `M1`, `M2`, ... consécutifs sans trou ;
- `sum(Losses)` sur une phase = nombre de matchs terminés (non annulés) de cette phase ;
- `remainingLives(p) = Lives[p] - Losses[p]`, borné à 0 ;
- un `GMatch Done` a `Winner` et `Loser` renseignés, sauf s’il est `Skipped` ;
- `resolve` est idempotente : l’appeler deux fois de suite ne change rien ;
- `Replay(journal)` est une fonction pure du journal.

18 Limites connues et extensions prévues

Reprise priorisée de `docs/RESTE_A_FAIRE.md`. Ces points ne sont **pas** implémentés : une reconstruction fidèle du moteur ne doit pas les inclure sans le dire.

18.1 Avant un premier tournoi réel

- **Interface TD dans le logiciel hôte** : le moteur ne fournit que `Propose / Apply`. Il faut une page listant les actions avec un bouton de confirmation par action, une saisie de résultat par match en cours, et les boutons correction / annulation / forfait. Contrat : un bouton = un événement ; vérification : `Replay` donne le même état, sans avertissement.
- **Persistance du journal côté hôte** (une ligne JSON par événement, en ajout). Tester une coupure en plein tournoi.
- **Validation manuelle** sur un tournoi de club de 16 à 32 joueurs, en doublant sur papier.
- **Règle anti-manipulation du continu** : aujourd’hui `Propose` apparie tous les joueurs libres à chaque appel ; l’hôte doit l’appeler à intervalle fixe pour obtenir un effet micro-rondes. À implémenter proprement : paramètre `batch_minutes` et horodatage du dernier lot dans `phase_swiss.go`.

18.2 Fonctions attendues d’un logiciel de tournoi

- **Forfaits fins** : il manque le forfait pour un seul match sans retrait, et le retrait « à partir de la ronde suivante ».

- **Pauses programmées** : Propose ne doit pas lancer un match dont la fin attendue dépasse l'heure de la pause. Paramètre `Config.Breaks []TimeRange`.
- **Équité des byes entre groupes** : la règle « pas de second bye tant que d'autres n'en ont pas eu » n'est appliquée qu'à l'intérieur d'un groupe de défaites.
- **Options de saut** pour la consolante et la dernière chance (perdants du tour 1 seulement, jusqu'au tour k , consolante à tirage avec heure limite d'entrée).
- **Têtes de série optionnelles (seeding)** : "rating" dans `drawSlots`, écartées en v1 par choix.
- **Classement des places non gagnantes** : règles à valider avec la FFBG ; codes traduisibles à substituer aux notes en français brut.
- **Prix** : structures en pourcentage du pool, retenue d'organisation, arrondi, prix séparés par section.
- **Exports** : feuille d'appariements imprimable par ronde ou par bloc ; import CSV avec ratings FFBG.

18.3 Moteur : robustesse

- **Correction d'un résultat de tableau après coup** : l'avertissement existe, la réparation n'est pas outillée (rejouer le match, annuler la suite en série).
- **Rematches dans les consolantes** : l'ordre inversé des drops n'évite que les rencontres immédiates. À mesurer par simulation.
- **Poules** : le cas d'égalité à trois pour deux places n'a pas été éprouvé sur un tournoi réel (durée du barrage).
- **Bascule Σ vies = 2^k en mode rounds** : la somme décroît par paquets et la ronde peut ne pas se lancer si elle ferait passer sous la cible ; à corriger dans le budget de `proposeSwissRound`.
- **API stable** : versionner le format du journal (champ `version` dans `Event`), documenter la compatibilité ascendante, fuzzer `Apply` sur des journaux aléatoires.
- **Rendu** : pas de tests (prévoir des fichiers témoins SVG/HTML), pas de vue double élimination côte à côte, pas d'écran joueur.

19 Annexes

19.1 A. Récapitulatif de l'API publique

19.1.1 Types

`PlayerID`, `Player`, `MatchID`, `MatchStatus`, `Match`, `Rank`, `ActionKind`, `Action`, `Draw`, `EventKind`, `Event`, `Journal`, `Config`, `PhaseConfig`, `Tables`, `TableRule`, `State`, `PhaseState`, `Section`, `GMatch`, `Src`, `Slot`, `Clock`, `External`, `ConfigRefusal`, `Absence`.

Codes (voir `codes.go`, aucun texte destiné à l'affichage ne sort du moteur) : `LabelKind`, `Label`, `NoteKind`, `Note`, `WarningCode`, `Warning`, `InfoCode`, `Info`, `ReasonCode` (`NoteWithdrawn` : retiré, classé sur son parcours ; `NoteForfeit` : match perdu par forfait).

19.1.2 Constantes

`BYE`, `Running`, `Finished`, `Cancelled`, `ActStartMatch`, `ActBye`, `ActDraw`, `ActNextPhase`, `ActFinish`, `ActWait`, `EvCreated`, `EvPlayerAdded`, `EvPlayerUpdated`, `EvPlayerUnavailable`, `EvPlayerAvailable`, `EvPlayerWithdrawn`, `EvMatchStarted`, `EvResult`, `EvResultCorrected`, `EvMatchCancelled`, `EvBye`, `EvDraw`, `EvNextPhase`, `EvLengthChanged`, `EvTableChanged`, `EvFinished`, `EvNote`, `KindSwissLives`, `KindLivesBracket`, `KindGSL`, `KindBracket`, `KindRoundRobin`, `JournalVersion` ; les codes `Label*`, `Note*`, `Warn*`, `Info*`, `Reason*`.

19.1.3 Fonctions et méthodes

```
// cycle de vie
func New(cfg Config, seed int64, now time.Time) (*State, Event, error)
func Replay(j Journal) (*State, error)
func (s *State) Apply(ev Event) error
func (s *State) Step(evs ...Event) ([]Action, error)

// boucle du TD
func (s *State) Propose() []Action // = ProposeAt(s.Last)
```

```

func (s *State) ProposeAt(now time.Time) []Action
func (s *State) ProposeWith(now time.Time, ext External) []Action // tables et joueurs pris par une au
func (s *State) EventFromAction(a Action, now time.Time) (Event, error)
func ResultEvent(id MatchID, winner PlayerID, scoreA, scoreB int, now time.Time) Event
func (s *State) CheckConfig(next Config) error // prévisualisation ; refus = *ConfigRefusal

// consultation
func (s *State) Running() []*Match
func (s *State) Ranking() []Rank
func (s *State) FreeSlots() []Slot // places d'exemption libres (retardataires)
func (s *State) StandingsCSV() []byte
func (s *State) SectionRanking(section string) []Rank
func (s *State) Pool() float64
func (s *State) Distributable() float64
func (s *State) PrizeAmounts(section string) []float64
func (s *State) SectionPrizes(section string) map[PlayerID]float64
func (s *State) Expected(n int) time.Duration
func (s *State) SlowMatches(now time.Time, facteur float64) []*Match
func (s *State) ClockAt(now, start time.Time) Clock
func Prizes(ranking []Rank, prizes []float64) map[PlayerID]float64

// sérialisation
func (j Journal) Bytes() ([]byte, error)
func ParseJournal(b []byte) (Journal, error)
func ParseConfig(b []byte) (*Config, error)
func (c *Config) Validate() error

// rendu français des codes (fr.go) - la console, la démo et render ; un hôte
// multilingue traduit les codes lui-même et ignore ce fichier
func (l Label) String() string
func (n Note) String() string
func (w Warning) String() string
func (i Info) String() string
func (r ReasonCode) String() string
func PhaseName(p PhaseConfig) string

// divers
func (m *Match) Loser() PlayerID
func (m *Match) Has(p PlayerID) bool
func (a Action) String() string

```

Tout le reste (pairGroup, drawSlots, resolve, bracketSection...) est **non exporté** : ce sont des détails d'implémentation que l'on doit pouvoir changer sans casser les journaux.

19.2 B. Exemple de journal

```

[
  {"seq":0,"kind":"created","time":"2026-01-01T09:00:00Z",
    "config":{"name":"Open de printemps",
      "phases":[{"kind":"swiss_lives","name":"Suisse 2 vies","lives":2,"length":7,
        "mode":"continuous","pairing":"random","target":16,
        "entry":"survivors"},
        {"kind":"lives_bracket","name":"Tableau final","length":9,
        "final_length":11,"entry":"survivors"}],
      "min_per_point":8},
    "seed":42},
  {"seq":1,"kind":"player_added","time":"2026-01-01T09:01:00Z",
    "player":{"id":"alice","name":"Alice Martin","club":"Paris","rating":4.2}},
  {"seq":2,"kind":"player_added","time":"2026-01-01T09:01:10Z",
    "player":{"id":"bob","name":"Bob Durand","club":"Lyon","rating":6.8}},

```

```

{"seq":3,"kind":"match_started","time":"2026-01-01T09:30:00Z","match_id":"M1",
 "label":"0 défaite(s), match 1","a":"alice","b":"bob","length":7,"table":1},
{"seq":4,"kind":"result","time":"2026-01-01T10:22:00Z","match_id":"M1",
 "winner":"alice","score_a":7,"score_b":4},
{"seq":5,"kind":"draw","time":"2026-01-01T14:00:00Z","phase":1,
 "draw":{"slots":["alice","BYE","bob","carole"],
 "lives":{"alice":2,"bob":1,"carole":1}}},
{"seq":6,"kind":"finished","time":"2026-01-01T18:40:00Z"}
]

```

19.3 C. Nomenclature des clés

Objet	Forme de la clé	Exemple
Match réel	M<n>	M17
GMatch de tableau	<section>.<tour>.<indice>	main.2.1, conso.3.0, last.0.2
GMatch de grande finale	gf.<n>	gf.0, gf.1
GMatch de groupe GSL	<section>.<n>	B2G3.4
GMatch de poule	<section>.<ronde>.<indice>	Poule A.1.0
Section de bloc GSL	B<bloc>G<groupe>	B2G3
Section de poule	Poule <lettre>	Poule C
Section de barrage	Barrage <poule>	Barrage Poule C

19.4 D. Documents liés

Fichier	Contenu
README.md	Présentation courte et mode d'emploi
doc.go	Documentation du paquet, exemple d'intégration
CLAUDE.md	Guide de contribution (commandes, architecture, conventions)
docs/etude_formats.md	Synthèse de l'étude par simulation qui justifie les choix de format
docs/RESTE_A_FAIRE.md	Liste priorisée du travail restant
exemples/index.html	Galerie de tournois simulés rendus à quatre stades, pour sept formats